

GUIA DE USO CONSCIENTE E ÉTICO DE FERRAMENTAS DE IA PARA GERAÇÃO AUTOMÁTICA DE CÓDIGO

Prof. Edinilson da Silva Vida

1 INTRODUÇÃO

A inteligência artificial vem transformando o desenvolvimento de software, inclusive na educação em Computação. Ferramentas como **Replit** e **Lovable** permitem criar aplicações completas (front-end e back-end) a partir de simples descrições em linguagem natural, atuando como “geradores automáticos de código”. Isso significa que um estudante pode descrever uma ideia de aplicativo ou site e a IA produz o código correspondente, um processo que antigamente exigiria horas de programação manual. Diante desse cenário, professores e alunos se deparam com desafios e oportunidades: como aproveitar esses recursos para **aprender lógica e programação** de forma eficaz, sem comprometer a autoria, a integridade acadêmica e o desenvolvimento de habilidades fundamentais? Este guia busca responder a essa pergunta de forma reflexiva e prática, oferecendo orientações para o uso **pedagógico e ético** de ferramentas de IA na geração de código em contextos de ensino técnico e superior em Computação.

Vamos explorar primeiro o funcionamento do Replit e do Lovable, depois discutir seus benefícios educacionais, os riscos e desafios éticos envolvidos, e apresentar estratégias para orientar os alunos no uso responsável dessas IAs. Também traremos exemplos práticos de aplicação em sala de aula e propostas de **avaliação diferenciada**, métodos para verificar o real conhecimento do estudante mesmo quando parte do código foi gerada por IA. Por fim, o guia oferece recomendações de boas práticas para professores e uma reflexão sobre o papel da IA na formação de desenvolvedores conscientes e críticos.

Reforçamos que este guia foca no uso pedagógico e ético das ferramentas, sem compará-las em termos de desempenho ou preferência; o objetivo é entender **como usá-las adequadamente no ensino**, e não determinar qual é “melhor”.

2 CONTEXTO E DESAFIOS DO USO DE IA EM PROJETOS DE PROGRAMAÇÃO

Ferramentas de inteligência artificial generativa como **ChatGPT**, **GitHub Copilot** e **Replit** estão cada vez mais acessíveis e sendo utilizadas por estudantes de programação. Esses modelos de IA são capazes de **gerar código-fonte a partir de descrições em linguagem natural**, muitas vezes com alta precisão e velocidade. Por exemplo, em um experimento, o modelo **OpenAI Codex** (base do Copilot) obteve desempenho equivalente ao quartil superior da turma em provas de programação introdutória, acertando cerca de 78% das questões.

Da mesma forma, alunos que utilizaram o GitHub Copilot em tarefas simples **resolveram mais problemas corretamente e em menos tempo** do que aqueles que codificaram sem essa ajuda. Esses casos ilustram que **tarefas tradicionais de programação podem ser resolvidas pela IA com facilidade**, o que levanta preocupações para educadores: se um estudante apresenta um projeto impecável em pouco tempo, como saber o quanto ele realmente compreende do código? Além disso, **ferramentas antiplágio convencionais nem sempre detectam código gerado por IA**, pois o algoritmo pode produzir soluções originais que não coincidem exatamente com fontes conhecidas.

Apesar desses desafios, proibir totalmente o uso de IA não é recomendável nem realista. Assim como calculadoras e outras tecnologias foram incorporadas no passado, a **IA generativa deve ser integrada de forma pedagógica e ética** ao ensino. Organismos internacionais e especialistas enfatizam a necessidade de equilibrar inovação com integridade acadêmica. A UNESCO, por exemplo,

lançou em 2023 um guia sobre ChatGPT no ensino superior destacando as **mudanças significativas e implicações éticas** trazidas por essas ferramentas. Esse guia aponta riscos centrais como a **integridade acadêmica, a falta de regulamentação e vieses nos conteúdos gerados**, preocupações que requerem respostas educativas, não apenas proibitivas. No Brasil, o Conselho Nacional de Educação (CNE) reconheceu a urgência do tema e está elaborando diretrizes para o uso da IA na educação, atualmente em debate público. Enquanto as diretrizes oficiais não se consolidam, cabe às instituições e professores adotar medidas próprias para garantir que o uso de IA **apoie** (e não prejudique) o aprendizado dos estudantes.

Em suma, vivemos um paradoxo: por um lado, a IA ameaça **distorcer a avaliação tradicional**, basta “aceitar a sugestão autogerada” pelo assistente de código para resolver muitos exercícios típicos; por outro, ela pode **potencializar o aprendizado** se usada conscientemente. O professor moderno precisa, portanto, **redesenhar estratégias de avaliação** que considerem a presença dessas ferramentas. A seguir, apresentamos um conjunto de estratégias para avaliar projetos de código feitos com auxílio de IA generativa, focando em aspectos-chave (complexidade do código, justificativas técnicas, personalização, adaptação, ética, documentação e autoria), bem como sugestões práticas para atividades que incentivem o uso consciente da IA. O objetivo é assegurar a **aprendizagem ativa e o desenvolvimento da autonomia e do pensamento computacional**, mesmo quando parte do código foi gerada por uma IA, mantendo a confiança na avaliação do real entendimento do aluno.

3 PRINCÍPIOS PARA INTEGRAÇÃO PEDAGÓGICA E ÉTICA DA IA

Antes de abordar as estratégias de avaliação em si, é fundamental estabelecer um **ambiente pedagógico propício** à utilização responsável da IA. O professor deve deixar claras as expectativas e limites do uso de ferramentas como ChatGPT e Copilot. Uma boa prática é **definir, juntamente com os alunos, diretrizes de uso de IA** na disciplina. Essas diretrizes podem incluir, por exemplo:

- A IA será usada apenas para apoiar o desenvolvimento intelectual, e não para substituí-lo.
- O uso de IA deve ser **transparente**: sempre declarar quais partes tiveram ajuda de ferramentas externas.
- É proibido submeter texto ou código gerado integralmente por IA sem a devida **atribuição ou revisão**.
- Devem-se seguir as orientações do professor sobre em quais etapas a IA é apropriada e quais etapas devem ser feitas manualmente.

Ao estabelecer um **“código de conduta” para uso de IA**, o docente promove uma cultura de **honestidade acadêmica** e uso consciente da tecnologia. Isso alinha-se a recomendações de especialistas: em sala de aula, **professores e estudantes devem permanecer no centro do processo educacional**, com a tecnologia servindo de apoio e não de substituto. A coordenadora de Educação da UNESCO no Brasil, Rebeca Otero, ressalta que é fundamental **salvaguardar a autonomia e o papel central do professor** frente à incorporação crescente da IA, garantindo que ele detenha o controle pedagógico sobre quando e como usar essas ferramentas. Em outras palavras, a **intencionalidade pedagógica** deve guiar o uso da IA: só se emprega a tecnologia se isso contribuir para os objetivos de aprendizagem planejados.

Outra consideração ética importante é a **equidade de acesso e preparação**. Nem todos os alunos terão familiaridade prévia ou acesso irrestrito a ferramentas de IA (muitas exigem cadastro, idioma inglês, ou versão paga). O professor pode mitigar isso apresentando em aula algumas dessas ferramentas, discutindo seus limites (por exemplo, mostrar que o ChatGPT pode gerar respostas convincentes, porém incorretas) e indicando alternativas gratuitas. Essa “nivelada” no letramento de IA garante que nenhum aluno seja privilegiado ou prejudicado por saber (ou não) usar essas tecnologias. Também é válido oferecer caminhos para alunos que, por motivos pessoais ou

filosóficos, optem por não usar IA, por exemplo, permitir que realizem determinada tarefa de forma tradicional, sem serem penalizados, ou que a utilização da ferramenta seja opcional e supervisionada.

Por fim, vale lembrar que **transparência e ética andam juntas**. Se um aluno empregou um gerador de código, é preferível que ele se sinta seguro para admitir isso e mostrar o que fez a partir do código sugerido, do que esconder o uso com receio de punição. Uma prática possível é solicitar que os alunos entreguem, junto com o projeto, um breve relato do processo de desenvolvimento, incluindo eventuais consultas a IA, trechos de código reutilizados de outras fontes etc. Essa prestação de contas demonstra maturidade ética e pode ser incorporada como critério de avaliação. De fato, **refletir sobre o uso da IA** faz parte do aprendizado: a UNESCO argumenta que é preciso formar estudantes e docentes quanto às **dimensões técnicas, éticas e pedagógicas** da IA, justamente para que possam navegar pelos desafios como viés nos algoritmos, confiabilidade das respostas e integridade acadêmica. Em resumo, integrar a IA de forma pedagógica requer **orientação clara, transparência, gradualismo e discussão ética** constante, criando-se um contexto em que a tecnologia agrega valor ao aprendizado sem comprometer os valores educacionais.

4 VISÃO GERAL DAS FERRAMENTAS: REPLIT E LOVABLE

Antes de tudo, é importante entender **como funcionam** as plataformas em destaque, Replit e Lovable, e quais recursos elas oferecem. Ambas permitem que mesmo usuários com pouca experiência obtenham código funcional a partir de comandos em linguagem natural, mas cada uma aborda essa ideia de maneira específica.

4.1 Replit: IDE online com assistente de código por IA

O **Replit** é conhecido como um ambiente de desenvolvimento integrado (IDE) totalmente baseado em navegador, suportando mais de 50 linguagens de programação e com colaboração em tempo real. Ele elimina a necessidade de configurar ambiente local: basta acessar o site e começar a codificar. Um dos diferenciais do Replit é o seu assistente de IA chamado **Ghostwriter**, essencialmente um “par programador” virtual integrado ao editor de código. O Ghostwriter oferece autocompletar inteligente, sugestões de código contextuais e até explicações de trechos, ajudando a escrever funções inteiras e a entender o que está sendo escrito. Por exemplo, enquanto o aluno digita, o Ghostwriter prevê e completa as próximas linhas com base no contexto do projeto. Além disso, o Ghostwriter pode detectar erros e sugerir correções em tempo real, funcionando como um depurador guiado por IA.

Outra funcionalidade recente do Replit é o **Replit AI Agent**, que permite criar aplicações completas via chat. O usuário descreve em linguagem natural o que deseja (como “quero um site de portfólio com galeria de fotos e formulário de contato”), e a IA gera automaticamente um projeto executável, com front-end e back-end, pronto para implantação. É “como ter uma equipe de engenheiros de software sob demanda”, segundo a própria Replit. Esse agente constrói o aplicativo e pode até corrigi-lo se encontrar bugs, tudo sem que o usuário precise escrever código manualmente. Vale notar, porém, que o Replit continua sendo uma IDE completa: após a geração inicial, o código fica disponível para o usuário inspecionar, editar e desenvolver conforme desejar, mantendo total flexibilidade e controle (o que é excelente do ponto de vista educacional, já que permite dissecar e modificar o resultado gerado). Em resumo, o Replit combina um ambiente de programação colaborativo e acessível com ferramentas de IA (Ghostwriter e Agent) que **aceleram a prototipagem e auxiliam no código**, sem deixar de lado a possibilidade de programação tradicional.

4.2 Lovable: geração de aplicações a partir de prompts em linguagem natural

O **Lovable** é uma plataforma focada em transformar descrições em aplicativos web completos, visando especialmente usuários não técnicos ou desenvolvedores que queiram um pontapé inicial rápido em seus projetos. Na prática, o usuário conversa com a IA descrevendo o que deseja, por

exemplo: “crie uma aplicação tipo agenda de eventos, estilo Notion, com modo escuro e pagamentos via Stripe”, e o Lovable gera todo o código necessário, incluindo interface front-end e lógica back-end. Em poucos minutos, obtém-se um protótipo funcional e hospedado na nuvem, que pode ser testado imediatamente. Uma característica importante é que o **código fonte gerado pertence ao usuário**: a plataforma permite exportar o projeto (como arquivo ZIP ou enviá-lo para um repositório no GitHub) para que o desenvolvedor possa inspecionar, editar e evoluir o código livremente. Isso evita o lock-in em um sistema proprietário, do ponto de vista educacional, é ótimo pois os alunos podem abrir o código gerado em sala e estudá-lo como fariam com qualquer projeto convencional.

O Lovable busca gerar código limpo e próximo de padrões profissionais, dando ênfase à qualidade e flexibilidade do resultado. O usuário pode iterar sobre o prompt caso queira ajustes: por exemplo, após ver a primeira versão do aplicativo, pode pedir “agora torne os botões arredondados” ou “adicione autenticação com Google”, e a IA **refina o código existente** em vez de recomeçar do zero. A plataforma conta com suporte a modelos de linguagem avançados (como Claude e GPT-4) para interpretações mais específicas dos pedidos. Em suma, o Lovable se destaca por entregar aplicações completas a partir de comandos em linguagem natural, com **foco na rapidez e na usabilidade do código gerado**. É ideal para prototipar ideias de forma quase instantânea, o que pode beneficiar tanto indie hackers quanto estudantes visualizando um projeto funcionando sem precisar construir tudo manualmente. No contexto educacional, isso significa que alunos podem experimentar construir sistemas web (com front-end, banco de dados, autenticação etc.) apenas descrevendo seus requisitos, e então estudar aquele código para entender como as peças se encaixam.

4.3 Como essas ferramentas funcionam?

Em linhas gerais, tanto Ghostwriter/Replit Agent quanto Lovable utilizam modelos de linguagem treinados em enormes bases de código. Eles recebem um prompt (um pedido textual) do usuário e produzem código fonte coerente com aquele pedido, usando seu conhecimento aprendido de exemplos. O processo não é perfeito, às vezes o código precisa de ajustes, mas, é impressionantemente eficaz para gerar uma **base inicial**. É como ter um assistente que escreve rascunhos de programas conforme você diz o que quer. Importante: o desenvolvedor (ou aluno) continua no comando para revisar, testar e aprimorar esse rascunho. Nos tópicos a seguir, discutiremos como aproveitar esse “assistente de programação” de forma proveitosa no ensino de Computação, e quais cuidados tomar.

5 POTENCIAIS BENEFÍCIOS NO ENSINO DE PROGRAMAÇÃO

Ferramentas de geração automática de código podem ser aliadas poderosas no aprendizado de programação, desde que usadas com objetivos pedagógicos claros. Abaixo, destacamos alguns **benefícios potenciais** de integrar plataformas como Replit e Lovable no ensino de lógica, linguagens e desenvolvimento de software.

5.1 Aprendizado acelerado de sintaxe e estruturas

Para iniciantes, um dos grandes desafios ao programar é lembrar a sintaxe exata e as estruturas de código necessárias para implementar determinada funcionalidade. Assistentes de código por IA, como o Ghostwriter no Replit, ajudam a preencher essas lacunas sugerindo trechos corretos e bem formados conforme o aluno digita. Isso pode reduzir a frustração com erros triviais de sintaxe e permitir que o estudante concentre-se em compreender o que o código faz, e não apenas como escrevê-lo caractere por caractere.

De fato, o Ghostwriter foi **projetado com iniciantes em mente**, chegando a explicar o código sugerido, o que o torna uma ótima ferramenta educacional para quem está aprendendo sozinho ou em sala. Essa assistência imediata funciona como um “apoio instrucional”: o aluno recebe feedback

instantâneo e exemplos práticos de implementação, o que reforça o aprendizado. Estudos indicam que ferramentas como ChatGPT/Copilot podem fornecer explicações passo a passo e ajudar a depurar erros, guiando o aluno em problemas complexos com respostas rápidas e contextualizadas. Isso equivale a ter um tutor disponível a qualquer hora, esclarecendo dúvidas no momento em que surgem.

5.2 Foco nos conceitos e lógica, aliviando a carga mecânica

Na perspectiva pedagógica, um grande ganho do uso de IA na programação é a possibilidade de **reduzir a carga cognitiva extrínseca** do aluno novato. Ao automatizar tarefas de boilerplate (código padrão repetitivo) e aliviar a necessidade de lembrar cada detalhe de sintaxe, a ferramenta de IA permite que o estudante direcione mais energia mental para os **conceitos lógicos e de resolução de problemas** em si.

Por exemplo, em vez de gastar toda a aula lidando com configurações de projeto ou escrevendo do zero funções comuns (como conexão a banco de dados, ou código HTML básico de formulário), o aluno pode obter isso gerado rapidamente e então **analisar a estrutura resultante**, entendendo por que aquelas partes são necessárias, como se relacionam etc. Isso está alinhado com princípios de educação: se a IA cuida do “braçal”, o professor pode guiar os alunos na parte importante, que é o porquê do código. Assim, a tecnologia atua como uma ferramenta de “andaime” no aprendizado, dá suporte nas partes maçantes ou muito difíceis de início, enquanto o aluno constrói sua competência nos fundamentos. Evidências sugerem que o uso orientado de IA pode aumentar a **autoeficácia e motivação** dos estudantes, pois eles conseguem progredir mais rápido e ver resultados tangíveis de seus programas, o que os encoraja a continuar aprendendo. Em outras palavras, ao conseguir criar um programa funcional (ainda que com ajuda), o aluno ganha confiança para explorar mais e se engajar com desafios de programação.

5.3 Visualização de exemplos e múltiplas soluções

Outra vantagem é a possibilidade de gerar **vários exemplos de código** rapidamente. Um professor pode, por exemplo, pedir ao Lovable para criar duas implementações diferentes de um mesmo aplicativo (como um to-do list: uma versão usando JavaScript com Node.js no back-end e outra usando Python/Flask) e, em seguida, pedir aos alunos que comparem as abordagens. Isso enriquece o entendimento, pois expõe os estudantes a diferentes maneiras de resolver o problema, diferentes estruturas de projeto, estilos de código etc.

De forma similar, um aluno pode utilizar a IA para experimentar soluções alternativas: “como seria se em vez de usar um loop eu tentasse um algoritmo recursivo?” A IA pode gerar a variação, e o aluno então estuda o novo código. Essa exploração rápida reforça a lógica de programação, porque o estudante vê na prática as consequências de certas decisões. Além disso, as ferramentas podem sugerir melhores práticas: por serem treinadas em grandes bases de código, muitas vezes geram soluções que seguem padrões reconhecidos. Por exemplo, se um aluno pedir para “criar uma função de ordenação”, o Ghostwriter pode produzir um código bem estruturado de quicksort ou mergesort, possivelmente mais organizado do que o aluno faria sozinho na primeira tentativa. **Aprender com esses exemplos** pode reforçar o senso de organização e estilo de programação do aluno (desde que se discuta em aula o porquê daquele código ser considerado bom, claro).

5.4 Ajuda na superação de obstáculos e depuração

Quantas vezes um estudante trava em um erro de sintaxe oculto ou em um bug lógico difícil de identificar? Nesses momentos, a IA pode atuar como um assistente de depuração. Ferramentas como o Ghostwriter possuem recursos de análise de erro, por exemplo, se o código não compila ou lança uma exceção, a IA pode sugerir onde está o erro e até propor a correção. Isso não substitui o

aprendizado de depurar manualmente, mas serve como uma rede de segurança que evita perda de tempo excessiva em um só ponto.

Pedagogicamente, o benefício está em manter o fluxo de aprendizagem: em vez de o aluno desistir por causa de um ponto-e-vírgula faltante ou uma variável mal nomeada, ele recebe uma dica, corrige e segue adiante, assimilando a correção no processo. Além disso, o aluno pode perguntar à IA “por que meu código não funciona?” ou “você pode testar esse código e ver se está correto?” e obter um tipo de feedback imediato que antes dependia totalmente do professor ou monitor. Esse feedback instantâneo, quando bem usado, **reforça o entendimento**, o estudante vê o erro, compreende a correção e aprende a evitar o mesmo engano futuramente. Vale mencionar também a utilidade na revisão: a IA pode atuar como um revisor de código, apontando melhorias de estilo, sugestões de refatoração e até alertando para possíveis vulnerabilidades de segurança em código mais avançado. Isso introduz os alunos a uma cultura de revisão contínua, comum na prática profissional.

5.5 Prototipagem rápida para projetos de criatividade

Em ambientes educacionais que valorizam projetos e criatividade (como hackathons, disciplinas de projeto integrador ou trabalhos de conclusão), ferramentas como Replit e Lovable permitem que os alunos materializem ideias de forma muito mais rápida. Em vez de gastar semanas aprendendo um novo framework só para testar uma ideia, eles podem descrevê-la à IA e obter um protótipo funcional em minutos.

Por exemplo, um grupo de alunos de um curso técnico que tenha uma ideia de aplicativo móvel simples pode usar o Bolt ou Lovable para gerar a estrutura básica e, a partir daí, focar seus esforços nas partes mais personalizadas ou inovadoras do projeto. Eles **aprendem vendo o projeto ganhar vida** rapidamente e podem iterar várias vezes dentro do semestre. Esse tipo de abordagem pode enriquecer o portfólio e a experiência prática, ao mesmo tempo em que ensina a importância de revisar e melhorar um código gerado automaticamente. Ademais, ao trabalhar com protótipos gerados por IA, os estudantes naturalmente esbarram nas limitações e precisam pensar criticamente: “o que o AI fez certo e o que preciso ajustar manualmente?”. Essa reflexão é extremamente educativa, pois ajuda a sedimentar conhecimentos (exemplo: “o login funciona, mas percebi que a IA não colocou verificação X, então aprendi que preciso adicionar essa verificação”).

Em resumo, quando bem empregadas, as ferramentas de IA para código podem **enriquecer o ensino de programação** ao fornecer suporte personalizado, acelerar o ciclo de aprendizado e expor os estudantes a códigos funcionais e boas práticas desde cedo. Pesquisadores destacam que essas IAs têm potencial para servir como auxiliares didáticos, ajudando alunos a superar obstáculos e dando feedback instantâneo, o que pode **intensificar a aprendizagem** de conceitos computacionais. No entanto, esses benefícios só se realizam plenamente se houver um direcionamento consciente, caso contrário, podem surgir problemas sérios, como veremos a seguir.

6 RISCOS E DESAFIOS ÉTICOS (AUTORIA, PLÁGIO E USO ACRÍTICO)

Junto com os benefícios, é fundamental reconhecer os **riscos e dilemas éticos** associados ao uso de IA na geração de código, especialmente em contextos acadêmicos. Professores e estudantes devem estar atentos a essas questões para que a adoção dessas ferramentas não comprometa a integridade do aprendizado. Destacamos alguns pontos críticos.

6.1 Autoria e plágio de código

Um dos desafios mais imediatos diz respeito à autoria do código gerado por IA. Se um estudante entrega um programa que foi, em grande parte, escrito pela IA a partir de uma descrição, até que ponto esse código pode ser considerado **trabalho original do aluno**? Do ponto de vista de

integridade acadêmica, apresentar código gerado automaticamente como se fosse 100% de sua própria lavra é problemático, equivale a entregar algo que não reflete completamente seu próprio esforço ou conhecimento, o que se aproxima de plágio. A situação se complica porque o código gerado é inédito (não foi simplesmente copiado de outra fonte específica), então a clássica detecção de plágio por comparação falha em apontar irregularidades. Alguns alunos podem pensar: “se a IA escreveu e não existe idêntico na Internet, então não é plágio.” Contudo, essa é uma interpretação perigosa, pois **continua sendo desonesto** apresentar trabalho alheio (no caso, de uma IA) sem créditos, mesmo que o conteúdo seja único.

Instituições educacionais vêm debatendo isso; já se fala em exigir que os alunos declarem se usaram IA e em que medida, para deixar claro o nível de ajuda externa. Professores enfrentam dificuldade crescente em determinar o quanto de um projeto é contribuição do aluno e o quanto veio pronto da ferramenta. Esse cenário exige novas abordagens de avaliação e conscientização: é preciso ensinar aos estudantes que usar IA não é “mágica sem pegadas”, e sim uma forma de auxílio que deve ser reconhecida e utilizada com transparência e ética.

6.2 Aprendizagem superficial e dependência excessiva

Outro risco é que o uso indiscriminado de geradores de código leve a uma **aprendizagem rasa**. Se o aluno se habituar a sempre pedir soluções prontas à IA para cada problema, ele pode acabar pulando etapas essenciais do pensamento computacional. Resolver um problema de programação não envolve apenas digitar código, mas todo um processo de compreender o enunciado, planejar a solução (em termos de algoritmos, estruturas de dados), implementar passo a passo e depurar. A IA pode entregar a implementação de imediato, mas se o estudante não entender o caminho que levou até ali, terá perdido a oportunidade de desenvolver **habilidades de resolução de problemas** e de **pensamento crítico** fundamentais na área.

Pesquisas alertam que a disponibilidade dessas soluções de IA pode “atrofiar” a capacidade do aluno de enfrentar desafios novos, pois ele se acostuma a buscar a resposta pronta em vez de refletir profundamente. Além disso, a prática de codificação vem acompanhada de aprendizado por tentativa e erro; quando a IA dá tudo certo de primeira (ou quase isso), o iniciante pode não vivenciar os erros necessários para aprender. Em suma, o estudante corre o risco de se tornar um **usuário passivo de código**, capaz de montar aplicativos por prompt, mas com compreensão limitada do funcionamento interno. Isso é perigoso não apenas academicamente (pelo conhecimento frágil), mas também profissionalmente, no mercado, esperar que uma IA resolva tudo pode levar a soluções inadequadas quando a IA falha ou quando surgem problemas inéditos. **Overreliance** (dependência excessiva) é um termo que descreve bem esse risco: a confiança cega na IA pode diminuir o desenvolvimento pleno das competências do programador em formação.

6.3 Qualidade e correção do código gerado

Embora impressionantes, as IAs de geração de código não são infalíveis. Elas podem produzir códigos com bugs sutis, vulnerabilidades de segurança, soluções ineficientes ou simplesmente incorretas para determinado contexto. Um estudante despreparado para avaliar criticamente o código pode aceitar a saída da IA como verdade absoluta e acabar entregando um programa com problemas (ou aprendendo conceitos errados).

Por exemplo, a IA pode usar uma abordagem que funciona nos casos mais comuns, mas falha em cenários de exceção; se o aluno não identificar isso, assumirá que a solução está 100% correta. Existe também o risco de a IA **reutilizar trechos de código de terceiros** presentes nos dados de treinamento, potencialmente infringindo licenças ou direitos autorais, ainda que raramente de forma

literal. Do ponto de vista ético, seria problemático incorporar código de uma fonte externa sem mencionar, mesmo que a fonte nesse caso seja difusa (o modelo de IA).

Portanto, confiar cegamente no código gerado é perigoso. É **imperativo revisar e testar cuidadosamente** qualquer código produzido pela IA antes de usá-lo em um projeto ou submetê-lo em uma avaliação. O professor deve alertar que a IA “parece saber tudo”, mas na verdade não garante correção: ela faz previsões com base em padrões e pode cometer erros. Aliás, erros da IA podem ser oportunidades de aprendizado, se o aluno for encorajado a encontrar e corrigir o erro, ele aprende ativamente. Mas se o erro passar despercebido, ele aprenderá o conceito de forma errada ou pensará que o problema está resolvido quando não está.

6.4 Questões de ética e honestidade acadêmica

No âmbito acadêmico, o **uso não declarado de IA** em trabalhos pode ser visto como uma forma de cola ou fraude, dependendo das políticas da instituição. Muitos cursos ainda estão estabelecendo diretrizes sobre isso. Há relatos de alunos enfrentando processos disciplinares por entregarem códigos “suspeitos” de serem gerados por IA, especialmente quando o resultado está muito além do nível esperado do estudante. Uma dificuldade é que detectar com 100% de certeza que um código veio de IA é praticamente impossível, há ferramentas que estimam probabilidades, mas nada conclusivo. Assim, cria-se um dilema: restringir totalmente o uso (o que pode ser contraproducente, visto que a IA está amplamente acessível), ou permitir com moderação? A posição mais equilibrada tem sido **permitir sob orientação** e avaliar a aprendizagem de formas complementares (veremos adiante estratégias de avaliação).

Independentemente da política, deve-se enfatizar a importância da honestidade: se o aluno usar a IA, que ele o faça de maneira responsável e, idealmente, cite ou reconheça a ajuda. Por exemplo, um estudante pode comentar no código: “// Função gerada com auxílio da ferramenta X, revisada pelo aluno”, normalizando a ideia de que usar a IA não é proibido, desde que **não mascare a falta de compreensão**. O pior cenário ético é aquele em que o aluno se torna apenas um “consumidor de respostas”, colando saídas da IA sem nenhum processamento próprio. Isso não só compromete a integridade acadêmica como **prejudica o próprio aluno**, que deixa de aprender. É importante conscientizar a turma de que usar IA sem critério é se enganar: pode render uma entrega rápida, mas cobra seu preço na hora de aplicar o conhecimento em provas, entrevistas de emprego ou situações práticas aonde a IA não estará disponível ou não será permitida.

6.5 Desafios na avaliação tradicional

Com a possibilidade de qualquer estudante gerar soluções impecáveis via IA, os métodos tradicionais de avaliação (listas de exercício para fazer em casa, projetos longos desenvolvidos sem supervisão etc.) tornam-se menos eficazes para medir o conhecimento real. Um risco é inflacionar notas ou certificações sem correspondência de competência, por exemplo, um aluno avança no curso sem realmente dominar programação, apoiado pelas pernas-de-pau da IA. Isso é um desafio para educadores, que precisam repensar **como avaliar**. Abordaremos adiante estratégias de avaliação diferenciada, mas vale mencionar aqui que uma consequência ética do mal uso dessas ferramentas é justamente a **quebra de confiabilidade nas avaliações**.

Professores podem sentir necessidade de voltar a práticas menos “autênticas” (como provas escritas, orais, vivas de programação) para ter certeza do conhecimento do aluno. Isso pode ser desgastante para ambos os lados se não for bem planejado. Portanto, do ponto de vista institucional e de curso, há um desafio ético em oferecer igualdade de condições: e se alguns alunos usam IA e outros não, como garantir justiça? Deve-se estabelecer diretrizes claras para que todos tenham as mesmas

oportunidades (por exemplo, ou todos podem usar nas tarefas X, ou nenhum; ou se usar, use de tal forma). A falta de comunicação clara pode levar a situações de “competição desleal” e ambiguidades.

Em resumo, os riscos vão desde **problemas de aprendizado** (não aprender de fato, viciar-se em respostas prontas) até **problemas de princípio** (plágio, desonestidade) e **desafios práticos** (avaliar conhecimento, detectar uso). Contudo, esses desafios **podem ser mitigados** com conscientização, orientação e adaptação das práticas de ensino. No próximo tópico, discutiremos como professores e alunos podem adotar **estratégias para o uso responsável** dessas ferramentas, de modo que elas sejam ajudantes do aprendizado, e não vilãs.

7 ESTRATÉGIAS PARA O USO RESPONSÁVEL (ALUNO COMO COAUTOR, NÃO CONSUMIDOR PASSIVO)

Para aproveitar as ferramentas de IA de maneira construtiva, é preciso **educar os alunos** a usá-las responsabilmente, em vez de serem meros consumidores passivos de código gerado, tornarem-se efetivamente **coautores** no processo. Abaixo estão algumas estratégias e orientações que professores podem adotar para guiar essa postura nos estudantes.

7.1 Deixe claro o papel da IA como ferramenta de apoio, não atalho para nota

Logo no início, discuta abertamente com a turma sobre o que significa usar IA em programação. Explique que a IA deve ser vista como um **auxílio para aprender e produzir melhor**, semelhante a um assistente, e não como uma forma de pular etapas ou “colar”. Coloque ênfase na ideia de coautoria: o aluno e a IA colaboram, mas o entendimento e as decisões de alto nível devem partir do aluno. Por exemplo, se um projeto é desenvolvido com ajuda do Ghostwriter, que o aluno tenha propriedade sobre ele, isto é, saiba explicar, modificar e justificar o código, tornando-se de fato coautor.

Reforce que **pedir código pronto e entregá-lo cegamente equivale a terceirizar o aprendizado**, o que vai contra os objetivos do curso. Muitos professores têm adotado políticas explícitas: permitir a IA somente se o aluno documentar seu uso e demonstrar compreensão do resultado. Uma boa prática é incluir no enunciado das tarefas algo como: “você pode utilizar ferramentas de IA como apoio, contanto que cite onde foram usadas e, principalmente, que **consiga explicar todas as partes do código**.” Isso já estabelece desde o início o uso responsável.

7.2 Use a IA para insights e explicações, não apenas respostas finais

Uma recomendação-chave (vinda inclusive de pesquisadores em educação) é orientar os alunos a explorar a IA como uma forma de **obter pistas e esclarecimentos**, em vez de simplesmente perguntar “me dê o código da solução X”. O professor pode demonstrar exemplos em sala: pegar um problema e mostrar duas abordagens de prompt, uma pedindo a resposta direta e outra pedindo orientação. Por exemplo: “resolva este exercício para mim” (errado) versus “estou tentando resolver assim e travado nessa parte, o que devo considerar?” (caminho mais orientado).

Mostrar ativamente essa diferença faz o aluno perceber que ele pode usar a IA para aprender mais. Conforme destacou um especialista, usar o ChatGPT apenas como um “vidente” que dá respostas prontas “destrói algo fundamental para a criatividade humana, que é a tomada de decisões corretas e de forma consciente”, enquanto empregá-lo como um **gerador de insights e novos caminhos** para solucionar um problema pode ser muito benéfico. Incentive os estudantes a fazer perguntas do tipo: “explique passo a passo como posso chegar na solução de tal coisa” em vez de “me dê o código pronto”.

Se um aluno está com um bug, ao invés de só pedir “conserte isso”, que tal solicitar: “quais são as possíveis causas desse erro e como posso depurar?”. Esse tipo de uso mantém o aluno no circuito do raciocínio. **Exemplo prático:** suponha que um estudante não entendeu recursão. Ele poderia pedir à IA: “poderia me dar um exemplo de função recursiva e explicar como ela funciona passo a passo?”. A resposta da IA servirá de explicação com código, quase um tutor personalizado. Assim o aluno aprende o conceito, e não apenas pega código para entregar. Em sala, o professor pode até propor exercícios onde o objetivo explícito é usar a IA para aprender, por exemplo, “use o Replit Ghostwriter para gerar uma função X e em seguida escreva, com suas palavras, como essa função funciona e por que o Ghostwriter a implementou desse jeito”. Isso força o uso consciente e evita a simples cópia.

7.3 Estimule a revisão crítica do código gerado

Deixe como **regra de ouro**: nenhum código saído da IA deve ser aceito sem passar pelo crivo do estudante. Ensine técnicas de revisão básica: ler o código linha a linha, adicionar comentários explicando o que faz cada trecho (excelente prática para ver se entendeu), testar com diferentes entradas, buscar pontos frágeis. Uma ideia é apresentar em aula alguns casos em que a IA errou (há vários exemplos documentados de erros do Copilot/ChatGPT em programação). Mostre um bug sutil e questione: “você teriam detectado isso? Como podemos evitar cair nessas pegadinhas?”. Isso cria uma **postura de validação** no uso da IA.

O aluno deve se sentir responsável pelo código, mesmo que não tenha digitado cada caractere. Lembre-os de que **o desenvolvedor humano continua indispensável** para verificar se a solução está correta, adequada e otimizada. Inclusive, incorporar ferramentas de análise estática ou testes automatizados junto com a geração de código é recomendado, por exemplo, depois que o Ghostwriter gera um trecho, rodar os testes (que podem ter sido escritos pelo aluno ou fornecidos pelo professor) para ver se tudo passa. Se não, usar isso como aprendizado: “ok, a IA não atendeu esse caso de teste, o que falta?”. Outra boa prática: sempre que a IA usar uma biblioteca ou função que o aluno desconhece, este deve pesquisar e entender do que se trata. Ou seja, **transformar a saída da IA em entrada de estudo**: “ela usou a biblioteca X, vou ler a documentação para entender por quê”. Com essa mentalidade, o aluno assume um papel ativo e a IA vira de fato uma ferramenta de aprendizagem.

7.4 Promova a transparência e ética no uso

Crie um ambiente onde os alunos se sintam confortáveis em dizer que usaram IA, sem medo de serem automaticamente penalizados, desde que tenham seguido as diretrizes combinadas. Uma sugestão é reservar momentos para discutir casos concretos: por exemplo, peça voluntários para contar como usaram a IA em uma tarefa e o que acharam do processo. Essa troca permite identificar quem talvez tenha usado de forma inadequada (e então orientá-lo) e também difundir boas práticas entre colegas. **Deixe claro que a IA não é “proibida” (a menos que a avaliação seja explicitamente sem consulta), mas que escondê-la do professor é desaconselhável.** Ao contrário, se o aluno proativamente diz “professor, usei o Lovable para gerar a base do front-end e depois eu ajustei o CSS manualmente”, isso pode ser visto positivamente, mostra iniciativa e honestidade, desde que ele saiba falar do resultado.

Uma prática interessante adotada em algumas disciplinas de nível superior é pedir, junto com a entrega do código, um breve **relato escrito do processo**, incluindo se utilizaram a IA, onde e por quê. Exemplo: “inicieei criando a estrutura no Replit com Ghostwriter, pois agilizou a configuração do projeto React; em seguida, desenvolvi os componentes personalizados manualmente. Usei o Ghostwriter novamente para me ajudar a corrigir um erro de integração com a API.” Esse tipo de reflexão escrita faz o aluno pensar sobre o próprio aprendizado e permite ao professor avaliar a metacognição e não apenas o produto. Ademais, discutir ética abertamente, como estamos fazendo,

é importante: fale sobre **plágio, atribuição e honestidade** no contexto de IA. Reforce que, fora da instituição de ensino, um desenvolvedor que depende cegamente de IA pode cometer deslizes graves (como violar licenças sem saber, introduzir vulnerabilidades de segurança etc.), então é no ambiente educativo que eles devem aprender limites e responsabilidades.

7.5 Incorpore a IA nas atividades de forma orientada

Uma estratégia inteligente é **integrar o uso da IA como parte do processo de aprendizagem planejado**, e não algo que os alunos usam à revelia. Por exemplo, o professor pode propor: “hoje faremos uma atividade de par-programming em trio: aluno, colega e Ghostwriter”. Cada dupla de alunos deve, sob orientação, utilizar o Ghostwriter para uma parte do exercício, e escrever o restante manualmente. Depois discutem qual foi a diferença, se o código da IA era compreensível, o que tiveram que ajustar. Assim, o uso é institucionalizado e supervisionado, garantindo que o foco permaneça educacional.

Outra ideia: **desafie os alunos a “ensinarem” a IA**, por incrível que pareça, ao tentar refinar prompts para obter o código desejado, o aluno está exercitando clareza de requisitos e decomposição do problema (habilidades fundamentais de programação). Promptar corretamente envolve pensar no problema de forma estruturada para explicar à IA, o que é em si um exercício de lógica. Valorize isso como parte do aprendizado de algoritmos: “escreva um prompt ótimo para fazer a IA gerar tal função” pode ser um mini-desafio. Tudo isso reforça que o aluno não é um simples espectador. Ele **conduz** a ferramenta, e não o inverso.

7.6 Destaque a importância da prática manual e fundamentos

Por fim, lembre sempre os estudantes de que, embora a IA possa ajudar a escrever código, ela não substitui a necessidade de entender os conceitos. Uma analogia útil: **calculadoras**. Em matemática, ninguém mais faz logaritmos na mão, usamos calculadoras ou software, porém, ensinamos o conceito de logaritmo, sua interpretação e verificamos se o aluno sabe quando e como aplicá-lo.

Da mesma forma, no futuro profissional, programadores usarão cada vez mais assistentes de IA, mas precisam ter a base sólida para saber se a resposta faz sentido e para moldar a solução adequada. Assim, motive-os a também praticar “na unha”: por exemplo, resolvendo alguns exercícios sem nenhuma ajuda para fixar conhecimento. Equilíbrio é a chave. Ter momentos “desplugados” (codificar em um quadro ou papel) pode ser interessante para consolidar lógica sem tentação de autocompletar. O aluno consciente saberá dosar: **usar a IA para se impulsionar, não para se carregar** completamente. Desenvolver esse discernimento é um dos objetivos formativos importantes ao se introduzir IA no curso.

Seguindo essas orientações, o estudante passa a ver a IA como uma **parceira de aprendizagem**. Em vez de minar sua autoria, ela expande suas capacidades, mas, a consciência e o controle permanecem com o aluno. Esse é o cenário ideal: formar profissionais que sabem colaborar com ferramentas inteligentes de forma ética e eficaz, mantendo espírito crítico. No próximo item, veremos exemplos concretos de como professores podem incorporar Replit, Lovable e similares em atividades e projetos, reforçando essa abordagem responsável.

8 EXEMPLOS PRÁTICOS DE USO PEDAGÓGICO DAS FERRAMENTAS

Para tornar mais palpáveis as ideias discutidas, vamos a alguns **exemplos práticos** de como Replit e Lovable podem ser incluídos em atividades de ensino, projetos ou desafios guiados. Os cenários a seguir ilustram abordagens didáticas aproveitando essas plataformas.

8.1 Projeto guiado com Lovable (explorando arquitetura gerada)

Imagine uma disciplina de Desenvolvimento Web em que os alunos estão aprendendo sobre a arquitetura de aplicações web (front-end, back-end, banco de dados). O professor pode propor um projeto guiado onde, na primeira parte, os alunos **utilizam o Lovable para gerar uma aplicação simples** a partir de um prompt bem definido, por exemplo: “crie uma aplicação de lista de tarefas (to-do list) com React no front-end, Node.js no back-end e um banco de dados SQL para persistência. Deve permitir cadastrar usuários e autenticar login.”. Em poucos minutos, cada aluno (ou grupo) terá uma aplicação funcional entregue pela IA. **A tarefa então passa a ser educativa:** os alunos devem analisar o código gerado. O professor pode fornecer um roteiro de análise: identificar os componentes front-end criados, os endpoints da API back-end, o modelo de dados etc. Os estudantes documentam (em um relatório ou apresentação) como o Lovable estruturou o projeto, por exemplo, “o front-end tem um componente principal App.jsx que lista as tarefas e utiliza um componente TaskItem para cada item; no back-end há um arquivo server.js configurando rotas REST para tasks e para autenticação, utilizando a biblioteca X; o banco de dados foi configurado usando SQLite com um schema Y;” e assim por diante.

Em seguida, vem a parte ativa: o professor pede que implementem uma nova funcionalidade sem ajuda da IA, usando o código gerado como base. Por exemplo: “agora acrescentem a possibilidade de marcar uma tarefa como concluída e filtrar por tarefas concluídas/não concluídas.” Os alunos então precisam ler e entender o código existente para modificá-lo adequadamente. Aqui eles aplicam lógica e codificação manual, mas com um projeto realista já montado. Ao final, cada grupo compartilha sua experiência, o que aprenderam inspecionando o código do Lovable, que mudanças fizeram, se encontraram alguma dificuldade ou erro gerado pela IA.

Essa atividade mostra como a IA pode **acelerar a obtenção de um protótipo**, mas a aprendizagem significativa ocorre na inspeção e extensão do código. Os alunos vivenciam um ciclo parecido com a vida real: pegar um código legado (no caso, gerado pela IA) e aprimorá-lo. De quebra, veem na prática a separação front/back e boas práticas possivelmente embutidas no projeto, discutindo sua finalidade.

8.2 Laboratório de resolução de problemas com Ghostwriter (foco em depuração e otimização)

Numa aula de Algoritmos, o professor pode planejar um laboratório onde os alunos vão resolver pequenos desafios usando o **Replit com Ghostwriter ativo**. Por exemplo, um desafio de programação clássica (como “ache os números primos até N” ou “ordene uma lista usando método X”). A orientação dada: cada aluno deve **tentar codificar a solução e pode interagir com o Ghostwriter para ajudá-lo** a chegar lá. Talvez o aluno comece escrevendo a função, e quando travar, aceite sugestões do Ghostwriter para completar. O importante é que ele veja as sugestões durante o processo, e não simplesmente peça a resposta final pronta.

Uma vez que o código esteja rodando, o professor insere a próxima fase: **depuração/otimização assistida**. Ele pode fornecer casos de teste mais difíceis ou limites maiores e pedir: “verifique se seu código funciona e é eficiente para N bem grande”, muitos alunos verão que talvez a solução precise de melhorias. É aí que entra novamente o Ghostwriter: incentive-os a perguntar via chat do Ghostwriter coisas como “como posso otimizar tal função?” ou “você vê algum problema de desempenho neste código?”. O Ghostwriter, tendo contexto do código, pode responder algo do tipo: “esta implementação tem complexidade quadrática, para otimizar você poderia usar técnica Y...”. Os alunos então tentam incorporar as sugestões e melhorar o código. Ao final do laboratório, reúne-se a turma para discutir: quem usou a IA para quê, qual dica foi útil, qual foi inútil ou incorreta? Por exemplo, um aluno pode dizer: “eu estava com dificuldade em eliminar números não primos de forma eficiente, o Ghostwriter sugeriu usar o Crivo de Eratóstenes e isso melhorou muito meu código”.

Outro pode apontar: “a IA me deu uma sugestão que não funcionou de primeira e tive que ajustar, aprendi que preciso sempre testar mesmo a sugestão”.

Essa troca reforça o aprendizado e normaliza o uso consciente da ferramenta. O professor pode complementar mostrando um **debug step-by-step** com IA: pegar um código com bug comum, colar no chat do Ghostwriter e perguntar “explique o erro aqui”. Assim os alunos veem como extrair explicações da IA (coisa que eles também podem fazer sozinhos para estudar). Esse laboratório torna a resolução de problemas mais dinâmica e interativa, mas mantém o aluno no controle e refletindo sobre as soluções.

8.3 Desafio de melhoria de código gerado (reflexão crítica)

Aqui o professor fornece deliberadamente um código gerado por IA (pode ser obtido previamente usando Replit ou Lovable) que **não está perfeito**, por exemplo, um código que “resolve” um problema, mas de forma ineficiente, ou uma aplicação web funcional, mas sem qualquer tratamento de erros ou com interface crua. Entrega esse código aos alunos como ponto de partida de um desafio: “melhore este código nos aspectos A, B, C”. Essa atividade coloca o aluno no papel de crítico e editor do trabalho da IA.

Por exemplo, suponha um código Python que ordena uma lista usando bubble sort gerado pela IA. O desafio pode ser: refatore para usar um método mais eficiente e explique por que é melhor. Ou um site gerado que não seja responsivo, peça para torná-lo responsivo. Os alunos terão que entender o código existente (novamente exercitando leitura e compreensão) e pensar em soluções. Eles podem inclusive usar a IA como auxílio nessa melhoria: perguntar “como posso tornar este código mais eficiente?” e então aplicar, sempre documentando o que foi feito. No final, uma exigência poderia ser um **relatório breve ou apresentação** indicando: quais problemas o código original tinha, como eles detectaram (testes, análise manual, ou ajuda da IA), o que foi modificado e por quê. Essa reflexão mostra se o aluno de fato compreendeu. Por exemplo: “o código original fazia três loops aninhados, o que é ineficiente. Identificamos que isso era desnecessário e substituímos por um único loop utilizando um dicionário para agilizar buscas. Com isso, reduzimos a complexidade de $O(n^3)$ para $O(n)$.”, uma resposta assim denota aprendizado.

Essa atividade ensina que a IA **nem sempre entrega a melhor solução** e que sempre há espaço para pensamento humano otimizar e aperfeiçoar. Ensina também a importante habilidade de code review: olhar um código escrito (ainda que por IA) e julgá-lo criticamente. Em termos éticos, reforça a mensagem de que **não se deve confiar cegamente**; em vez disso, deve-se questionar e melhorar o que a IA produzir.

8.4 Pair programming humano, IA e apresentação oral

Forma-se duplas de alunos. Cada dupla tem a missão de desenvolver uma pequena aplicação ou resolver um problema, mas com a seguinte dinâmica: um aluno atua como “piloto” humano escrevendo parte do código, o outro atua como moderador/solicitante com a IA (via chat do Ghostwriter ou similares), e alternam esses papéis.

Por exemplo, em um problema de implementar um jogo simples, o aluno A começa estruturando o código-base manualmente enquanto o aluno B pensa em como a IA poderia ajudar (talvez pedindo trechos específicos como “gerar função que verifica condição de vitória”). Depois trocam: B digita código manual enquanto A utiliza a IA para outra parte. Assim, ambos experimentam colaborar com o colega e com a IA. Toda a interação com a IA deve ser documentada (podem manter o log do chat). Ao final, cada dupla apresenta para a turma: mostram o resultado, explicam qual parte fizeram manualmente e qual parte a IA fez, e respondem perguntas do professor e colegas. Por exemplo:

“vejo que vocês usaram uma função X bem complexa, quem teve essa ideia?”, eles poderiam responder: “essa veio da IA quando pedimos uma otimização, nós nem conhecíamos essa abordagem e então fomos estudar para entender e incorporar.”.

Essa apresentação oral funciona como validação: se eles conseguirem responder satisfatoriamente às perguntas sobre o código, significa que internalizaram o conhecimento, mesmo tendo tido ajuda. Se travarem em algo fundamental, fica evidente que aquela parte não foi assimilada, e o professor pode intervir e esclarecer. Essa atividade fomenta trabalho em equipe e distribuição responsável de tarefas com IA, semelhante a times profissionais que usam IA como auxílio. Os alunos aprendem uns com os outros e perdem o receio de admitir o uso da IA, pois fez parte oficial do processo.

8.5 Utilizando Replit em projetos de classe invertida

O professor pode adotar o Replit para organizar projetos práticos. Por exemplo, em um módulo de estrutura de dados, dividir a turma em grupos e cada grupo, em casa, usa o Replit para implementar um tipo de estrutura (um grupo faz lista ligada, outro árvore binária etc.), eles podem usar o Ghostwriter para acelerar a implementação. Como o Replit é colaborativo (multiplayer em tempo real), os membros do grupo podem programar juntos remotamente, e até pedir ajuda ao Ghostwriter dentro desse ambiente. O professor pode entrar nos repositórios Replit dos grupos para acompanhar o progresso (vendo histórico de versões, quem editou o quê etc.).

Em sala de aula, na aula invertida, cada grupo apresenta sua estrutura de dados implementada, testando ao vivo, e então rodam um pequeno quiz prático: cada grupo desafia outro a usar sua estrutura (por exemplo, inserir valores na árvore, remover da lista) para ver se todos entendem.

Nesse formato, a IA foi uma ferramenta de produção fora da sala, mas a **validação ocorre dentro da sala** de forma interativa. Se algum grupo simplesmente copiou da IA sem entender, ficará nítido quando surgirem perguntas ou se algo quebrar nos testes ao vivo. Esse tipo de uso reforça a ideia de que o aprendizado será verificado, portanto usar IA sem aprender não adianta.

Os exemplos acima são apenas algumas ideias. O ponto em comum entre eles é: a IA é inserida como parte do processo educacional, nunca como substituta completa dele. Há sempre um momento em que o aluno precisa analisar, modificar, explicar ou complementar o código, garantindo engajamento cognitivo. O professor, por sua vez, atua criando estruturas e desafios que induzam o pensamento crítico mesmo com a comodidade da IA disponível.

8.6 Comparação entre solução manual e solução da IA

Proponha uma tarefa em duas etapas: primeiro, cada aluno tenta resolver um pequeno problema de programação **sem auxílio da IA** (podendo ser em sala, com tempo limitado); depois, permite-se que usem uma ferramenta de IA para obter outra solução para o mesmo problema. Em seguida, peça que **compare as duas soluções** por escrito: qual está mais correta ou eficiente? O que a IA fez diferente do que ele fez? Houve algo que o aluno fez melhor que a IA ou vice-versa? Essa atividade metacognitiva obriga o estudante a analisar código de forma comparativa, aprofundando sua compreensão. Ao ver a solução da IA, ele pode aprender um novo método; ao justificar a própria abordagem em relação à da máquina, ele consolida seu raciocínio.

Importante: deixe claro que não há demérito se a solução do aluno for inferior, o objetivo é o aprendizado. Essa dinâmica também desmistifica a IA, mostrando suas limitações. Alguns autores observaram, por exemplo, que mesmo modelos avançados podem cometer erros de cálculo ou

interpretação em questões conceituais, exigindo intervenção. Ou seja, **IA não significa infalibilidade**, e quem a usa precisa ter fundamentos sólidos para revisar o que ela produz. A atividade de comparação deixa isso evidente e encoraja o aluno a **confiar também no próprio raciocínio**.

8.7 Exercícios de engenharia de prompt

Uma abordagem inovadora é tratar a interação com a IA como uma habilidade a ser desenvolvida. Proponha mini-desafios de **“engenharia de prompt”**: divida a turma em grupos e dê a todos um mesmo problema de programação simples. Cada grupo deve formular diferentes perguntas ou comandos ao ChatGPT/Copilot tentando obter a melhor solução possível. Depois, compara-se em plenário as respostas obtidas e quais prompts foram mais eficazes. Os alunos logo percebem que a **qualidade da resposta da IA depende criticamente da qualidade da pergunta/prompt**, uma lição valiosa sobre especificação de problemas (que é parte do pensamento computacional).

Por exemplo, um prompt genérico “escreva um programa para ordenar números” talvez gere uma solução trivial, enquanto “escreva em Python uma função merge_sort recursiva para ordenar uma lista de inteiros e explique seu funcionamento” resultará em um código mais complexo e comentário passo a passo. Os grupos podem explorar estratégias: fornecer exemplos no prompt, pedir otimizações, esclarecer restrições. Ao avaliar, destaque a criatividade e a precisão nos prompts dos alunos. Essa atividade, além de envolver gamificação sadia, **ensina a pensar sobre os problemas de forma descritiva e estruturada**, afinal, para instruir a IA corretamente, o aluno precisa entender muito bem o problema ele mesmo.

Também prepara para um futuro em que “saber programar” pode incluir “saber conversar com o assistente de programação”. Determinados autores sugerem que exercícios de prompt podem se tornar um novo tipo de prática em cursos de programação, para alinhar o ensino à era da IA generativa. Nesse sentido, o professor age como facilitador para que os estudantes aprendam a extrair o melhor da ferramenta, sem abrir mão da compreensão.

8.8 Requerer pseudocódigo e planejamento prévio

Para manter o foco no **pensamento computacional** e não apenas na sintaxe de código (que a IA domina bem), uma boa prática é exigir etapas de planejamento antes da codificação. Por exemplo, peça que cada aluno entregue um **pseudocódigo estruturado ou fluxograma** da solução antes de escrever o código final. Somente após aprovação desse planejamento (ou feedback sobre ele) ele partiria para a implementação em linguagem de programação, podendo ou não usar IA para acelerar.

Isso assegura que **a solução nasce da cabeça do aluno**, com clareza lógica, mesmo que depois ele aproveite sugestões automatizadas para detalhes sintáticos. Na correção, avalie a qualidade do pseudocódigo independente do código final. Idealmente, o pseudocódigo deve corresponder ao que o programa faz; discrepâncias gritantes podem indicar inserções de última hora que não passaram pelo crivo do planejamento (talvez fruto de intervenção da IA).

Com essa técnica, mesmo que o código final venha mais polido graças ao Copilot, a essência (o algoritmo em si) foi concebida pelo aluno, o que é pedagógico. Além disso, escrever pseudocódigo

desenvolve a habilidade de **abstrair e estruturar problemas**, componente fundamental do pensamento computacional e da autonomia do programador. Ferramentas de IA não fornecem isso prontamente; é algo que o estudante precisa exercitar. Portanto, ao estruturar atividades, incluir um design report ou pseudocódigo inicial como entregável (e parte da nota) é altamente recomendável.

8.9 Avaliações complementares individuais

Nenhuma estratégia que permita IA em projetos será completa sem algum mecanismo de verificação individual de conhecimento. Assim, continue aplicando **provas teóricas/práticas sem auxílio** (em sala, no laboratório com supervisão ou até orais) que contemplem os mesmos conteúdos dos projetos. Por exemplo, se os projetos envolvem estruturas de dados, faça uma prova em que o aluno trace o resultado de uma função de árvore binária a mão, ou escreva um trecho de código simples no papel.

Esses momentos tradicionais garantem que o estudante tenha a chance (e o incentivo) de consolidar o que aprendeu. Notas de provas e de projetos podem ser balanceadas de forma que um projeto excelente feito com “cola” de IA não salve o aluno que não demonstra conhecimento básico na prova. Pelo contrário, a discrepância serviria de alerta.

Algumas instituições têm adotado **avaliações orais relâmpago**: sorteia-se aleatoriamente alguns alunos na aula para explicar um conceito ou resolver um pequeno problema no quadro, valendo pontos de participação. Isso, aliado à possibilidade de um projeto ser escolhido aleatoriamente para uma defesa mais aprofundada, forma uma rede de segurança para manter os estudantes engajados em realmente aprender, e não só em fazer funcionar.

A literatura de ensino já aponta que diversificar métodos de avaliação é eficaz para prevenir fraudes e melhorar a aprendizagem. Com IA, essa diversificação se torna crítica. Portanto, use projetos assistidos por IA **como parte de um repertório maior de avaliações**, compondo com exercícios manuais, quizzes conceituais, apresentações etc. Assim, o aluno entende que a ferramenta é bem-vinda para o **aprendizado**, mas na hora de demonstrar competência ele precisará contar também com o próprio conhecimento.

8.10 Exploração guiada da IA em sala de aula

Em vez de proibir o ChatGPT ou Copilot, apresente-os pedagogicamente. Por exemplo, projete em aula uma pergunta de programação no ChatGPT e analise criticamente a resposta junto com os alunos: ela está correta? Tem erros ou omissões? Quais? Essa atividade transforma a IA em objeto de estudo.

Os estudantes aprendem a não aceitar cegamente tudo que a ferramenta fornece, praticam identificação de erros e discutem possíveis melhorias. Isso reforça pensamento crítico e mostra na prática que a IA, apesar de poderosa, **pode falhar** e precisa de validação humana. Uma variante é o professor entregar um código **gerado pela IA com alguns defeitos sutis** e desafiar a turma a encontrar e corrigir esses defeitos (espécie de debugging challenge). Assim, eles exercitam depuração

e leitura de código, habilidades centrais do pensamento computacional, ao mesmo tempo em que interagem com produção da IA de forma ativa.

8.11 Projeto com peer review e discussão

Estruture os projetos de forma que haja algum nível de **revisão ou apresentação entre pares**. Por exemplo, após a entrega, organize sessões em que grupos de alunos trocam seus códigos e avaliam mutuamente aspectos como clareza, funcionalidade e possíveis sinais de uso desleixado de IA (como inconsistências nos comentários). Cada aluno pode apresentar o projeto de um colega, explicando-o como se fosse seu, o colega então complementa ou corrige.

Essa dinâmica garante que **ambos precisam entender profundamente o código**, seja para explicá-lo, seja para corrigir a explicação. Além disso, a revisão por pares pode incentivar conversas sobre estratégias: “você usou Copilot aqui? Eu tentei e veio tal resposta...”. Esse intercâmbio, mediado pelo professor, ajuda a difundir boas práticas de uso da IA e a alertar para ciladas (como código errado que a IA sugeriu e o aluno teve que consertar, por exemplo). Ao final, faça um debate aberto: como a IA apareceu nos trabalhos? Ajudou mais ou atrapalhou? O que aprendemos sobre programação que não teríamos aprendido se não revisássemos manualmente? Tais perguntas estimulam **pensamento crítico coletivo** e deixam claro que, mesmo com IA, é o conhecimento do programador que valida o resultado.

9 CRITÉRIOS PARA AVALIAÇÃO DIFERENCIADA

Dado o uso dessas ferramentas, é natural que a **avaliação do aprendizado** precise ser adaptada. Felizmente, existem estratégias de avaliação que podem medir o nível de conhecimento do estudante mesmo que parte do código apresentado tenha sido gerada por IA. O objetivo é assegurar que o aluno compreende o conteúdo e consegue aplicá-lo, ao invés de apenas verificar se ele consegue produzir um programa funcional sozinho. Abaixo listamos alguns critérios e métodos de avaliação diferenciada que podem ser combinados conforme o contexto.

9.1 Análise oral ou escrita do código gerado

Uma prática bastante eficaz é incluir, após a entrega de um projeto ou exercício, uma etapa de **explicação pelo aluno**. Por exemplo, o professor pode realizar pequenas sessões individuais (ou em dupla) de entrevista técnica, onde cada aluno traz seu código (pode até ser um print ou aberto em um laptop) e o professor faz perguntas como: “explique o que esse módulo faz e por que você o organizou assim” ou “qual é a finalidade dessa função? Poderia ter feito de outro jeito?”. Alternativamente, se o número de alunos for grande, pode-se solicitar uma explicação escrita: um anexo ao trabalho em que o estudante escreve brevemente o funcionamento de cada parte, talvez respondendo a perguntas guia. O importante é que o aluno demonstre **entendimento do código que entregou**, seja ele próprio escrito ou parcialmente gerado pela IA.

Essa abordagem desalenta o uso irresponsável da IA, pois mesmo que o código esteja impecável, o estudante terá que prestar contas do conteúdo. Professores relatam sucesso com algo semelhante a um testemunho oral de programação: permitir a consulta e até o código impresso, mas na avaliação o aluno precisa **narrar e justificar** seu programa. Uma sugestão oriunda da comunidade acadêmica é justamente fazer o aluno trazer uma cópia do código e explicar o que ele faz e por que foi construído daquela maneira. Essa explicação pode revelar muito: se foi tudo IA e o aluno não estudou, ele vai tropeçar nas justificativas ou não saberá certos termos do código. Já se ele realmente usou como

coautor, saberá comentar cada escolha. Essa técnica valoriza a capacidade de comunicação e reflexão, habilidades importantes para desenvolvedores conscientes.

9.2 Solicitação de modificações pontuais em tempo real

Esse método coloca o aluno em uma situação de hands-on supervisionado. Por exemplo, durante uma apresentação de projeto ou numa prova prática em laboratório, o professor pede ao aluno para fazer uma **pequena alteração no código entregue**. Poderia ser: “ok, seu programa faz X. Agora altere tal coisa para que ele também faça Y” (algo relativamente simples, compatível com o escopo do trabalho). Se o aluno desenvolveu e/ou entendeu seu código, conseguirá fazer a modificação ou pelo menos saberá por onde começar. Se não tem ideia de como o código funciona (porque, digamos, foi copiado da IA sem reflexão), ficará claro pela hesitação ou tentativa confusa.

Essa técnica pode ser aplicada de forma aleatória: o professor não precisa fazer todos os alunos alterarem algo, mas pode sortear alguns para verificar, só o fato de existir essa possibilidade já incentiva todos a estarem preparados. Outra variação é: diante do código do aluno, apresentar um novo requisito ou um bug hipotético e pedir “como você implementaria isso?” ou “onde no código você trataria essa nova condição?”. É quase como um mini coding interview dentro do contexto acadêmico. Inclusive, já há professores de computação implementando “coding interviews” regulares com os alunos para testar a compreensão de suas entregas. Essa forma de avaliação **valoriza o raciocínio em cima do código**, e não apenas a memória ou a autoria exata. O aluno demonstra seu nível de domínio ao conseguir adaptar ou corrigir o código sob demanda. Além de avaliar, isso reforça o aprendizado, pois obriga o estudante a pensar ativamente sobre o código. Importante: deixe claro à turma que essa dinâmica visa ajudá-los a **comprovar competência**, não é “pegar erro”. Quando bem conduzida, os alunos veem como um desafio positivo, quase gamificado, de mostrar que sabem o que fizeram.

9.3 Justificativa das escolhas de arquitetura ou estilo

Outra forma de diferenciar a avaliação é exigir que o aluno forneça **racionalizações para as decisões tomadas no código**. Por exemplo, se no projeto ele usou uma determinada estrutura de dados, ou separou o código em determinados módulos, ou estilizou a interface de certo modo, ele deve conseguir responder “por quê?”. Essa justificativa pode ser cobrada oralmente (durante uma banca ou apresentação) ou em relatórios que acompanham o código. A ideia é semelhante à anterior, mas foca nas decisões de design do software.

Se um aluno entrega um aplicativo gerado pela IA, possivelmente haverá decisões ali que ele nem pensou, cabe a ele então, se usou conscientemente, investigar e entender essas decisões a ponto de defendê-las. Por exemplo: “notei que meu back-end estava usando o framework Express.js com certo middleware; eu não escolhi ativamente, mas entendi que isso foi incluído para lidar com CORS e concordo que é necessário.”. Esse tipo de resposta mostra honestidade e aprendizado. Já uma resposta como “ah, não sei por que tem isso aí, mas apareceu...” denuncia uso acrítico.

O professor pode, durante uma apresentação, perguntar coisas como: “por que você optou por usar recursão aqui em vez de laço?” ou “vejo que seu front-end usa a biblioteca X, quais vantagens ela traz nesse contexto?”. Em avaliação escrita, poderia incluir questões reflexivas: “qual parte do código você achou mais complexa e como resolveu? Justifique a abordagem escolhida para a solução.”.

Há relatos de docentes que pedem até vídeos dos alunos explicando seu código, como forma de evidenciar autoria e entendimento. Por exemplo, gravar um vídeo curto estilo code walkthrough onde o aluno navega pelo código e comenta as partes principais e as escolhas. Isso não só assegura compreensão, como enriquece a habilidade de comunicação técnica do aluno. A justificativa de

escolhas também abrange aspectos de estilo: convenções de nome de variáveis, organização de pastas, formatação do código. Tudo isso o aluno deve estar atento, e caso algo venha da IA, ele precisa avaliar se está de acordo ou se ele mesmo adaptou. Essa postura crítica em relação a escolhas de design é justamente o que diferencia um desenvolvedor consciente de um mero usuário de ferramentas.

9.4 Reflexão crítica sobre o aprendizado obtido com a IA

Um critério muito valioso, embora não tradicional, é pedir ao aluno uma **reflexão metacognitiva**, ou seja, que ele próprio analise o quanto e o que aprendeu ao usar a IA no desenvolvimento do trabalho. Isso pode ser feito via perguntas específicas integradas à avaliação. Por exemplo: “descreva quais partes do projeto você gerou com auxílio de IA e o que aprendeu com cada uma dessas interações. Houve algo que a ferramenta fez que você não entendia totalmente? Como você procurou entender?”. Ou ainda: “comparando o desenvolvimento com IA e sem IA, quais foram as vantagens e desvantagens que você percebeu em termos de aprendizado pessoal?”. Essas perguntas forçam o estudante a **pensar sobre seu processo** e reconhecer ativamente o papel da IA. Ao ler as respostas, o professor consegue identificar níveis de profundidade: um aluno que responde superficialmente ou tentando esconder o uso (“ah, não usei muito, foi tranquilo”) possivelmente não se engajou; já um que relata “a IA me ajudou a estruturar o projeto inicialmente, aprendi a organizar melhor as camadas assim. Porém, percebi que ela não comentava o código, então tive que entender e comentar eu mesmo, o que consolidou meu entendimento” demonstra um aprendizado real e consciente.

Essa reflexão também serve para educar: quando os alunos colocam em palavras o que aprenderam (ou deixaram de aprender) usando a IA, eles próprios percebem as implicações. Pode emergir, por exemplo, a admissão de que “se eu não tivesse parado para estudar o código gerado, não teria aprendido tal conceito”, o que reforça a mensagem pedagógica. Avaliar essa reflexão dá peso acadêmico à forma de trabalho, não só ao resultado. Além disso, valoriza o esforço daqueles que usaram a IA de forma inteligente (aprendendo com ela), distinguindo-os daqueles que usaram de forma preguiçosa.

Em termos práticos de nota, o professor pode alocar uma porcentagem da pontuação do projeto a itens como “clareza da explicação/justificativas” e “percepção crítica sobre o próprio processo de desenvolvimento”. Isso comunica aos alunos que **não basta entregar um código rodando**, é preciso demonstrar que o conteúdo passou pela mente e não apenas pelo teclado deles.

9.5 Avaliações complementares de conhecimento

Embora não seja um critério específico, vale mencionar que as ferramentas de IA reforçam a necessidade de diversificar tipos de avaliação. Assim, provas teóricas curtas, testes de papel-e-caneta sobre trechos de código, quizzes em sala ou plataformas online e outras formas tradicionais podem continuar tendo espaço, mas focando mais em conceitos do que em pedir para escrever código extenso.

Por exemplo, perguntar “o que faz este código?” ou “qual a saída do algoritmo X para tal entrada?” avalia compreensão e não pode ser delegada à IA durante a prova. Da mesma forma, desafios práticos em aula (sob supervisão, sem acesso à Internet) podem avaliar a habilidade de lógica pura, aí todos estão em igualdade. Muitas instituições têm adotado a regra: o aluno só passa se também demonstrar desempenho mínimo em avaliações presenciais/tradicionais, independentemente dos projetos. Isso evita que alguém passe apenas com base em trabalhos feitos “com ajuda excessiva”. Não precisamos abandonar projetos (que são ótimos para aprendizagem), mas podemos **complementá-los** com estas verificações.

9.6 Personalização e criatividade no projeto

“Um dos principais riscos da IA é a uniformização e a perda das singularidades”, alertou João Brant, secretário de políticas digitais da Presidência, referindo-se à tendência de sistemas de IA basearem decisões em padrões médios e soluções genéricas. No contexto de projetos de programação, isso significa que se todos os alunos pedirem ajuda a um mesmo modelo para um mesmo problema, é provável que muitas respostas sigam um formato similar. Para **evitar essa homogeneização e estimular a autoria**, incentive a personalização dos projetos.

O professor pode **oferecer liberdade para cada aluno imprimir um toque pessoal** em sua aplicação, seja escolhendo o tema, adicionando funcionalidades extras a seu critério ou resolvendo um subproblema específico de interesse próprio dentro do escopo geral. Por exemplo, num projeto de banco de dados, além dos requisitos básicos, cada aluno poderia escolher um domínio de aplicação (esportes, música, biblioteca etc.) e incluir ao menos uma funcionalidade original relacionada a esse domínio. Ao avaliar, busque por esses elementos de criatividade: **código idêntico ou excessivamente padrão sugere pouco envolvimento pessoal**. Já um projeto que contenha funcionalidades únicas ou soluções incomuns (mesmo que imperfeitas) indica que o aluno realmente trabalhou no desenvolvimento, usando a IA talvez como suporte, mas não para todo o trabalho.

A personalização também aumenta o engajamento do aluno, pois ele se sente dono da ideia. Do ponto de vista avaliativo, projetos mais variados dão ao professor uma visão melhor de como cada estudante pensa e resolve problemas, em vez de receber 30 trabalhos quase iguais. Outra ideia é permitir que os alunos escolham qual problema resolver (dentro de um leque oferecido) ou definam parte dos requisitos. Isso naturalmente produz resultados diferentes e **reduz a chance de uma resposta única da IA servir para todos**. Portanto, ao corrigir, verifique: o aluno inseriu alguma **característica própria ou inovação** no código? Ele comentou alguma ideia experimental que testou? Valorize essas iniciativas, pois elas demonstram independência e capacidade de aplicação do conhecimento em contextos novos, habilidades que a simples automação não entrega.

9.7 Documentação e qualidade do código

Tradicionalmente, projetos de programação incluem critérios de avaliação como clareza do código, organização em funções/módulos, indentação correta, nomes de variáveis significativos e documentação. Esses aspectos **continuam válidos e ganham novo peso** no contexto de IA. Um código que venha todo desestruturado, sem comentários e com nomes confusos pode indicar “código de máquina” não filtrado pelo entendimento humano. Já um aluno que se preocupa em **comentar seu código, escrever um README explicativo e manter um padrão de formatação** provavelmente compreendeu e tomou posse do código, ainda que possa ter recebido sugestões automatizadas.

Portanto, mantenha no roteiro de avaliação pontos para **documentação interna e externa**: comentários explicando trechos não triviais, descrição da solução adotada, instruções de uso, etc. Inclusive, pode-se **exigir que o aluno documente quais partes do código foram geradas com ajuda da IA**, se for o caso, por exemplo, um comentário no início de uma função: “// Função sugerida pelo GitHub Copilot, revisada pelo aluno”. Essa atitude de atribuição é educativa e reforça a honestidade (embora ainda pouco usual, alguns cursos adotam essa prática).

Na correção, compare a documentação com o código: ela está consistente? O aluno consegue explicar em palavras o que o código faz? Se a documentação for inexistente ou superficial, e o código for sofisticado, é um sinal de alerta sobre possível uso negligente de IA. Por outro lado, uma **boa documentação agrega valor didático**: permite ao professor entender o raciocínio do aluno por trás daquele código. Em termos de qualidade, analise também aspectos como tratamentos de erro, cobertura de casos extremos, eficiência da solução e conformidade com o que foi ensinado em aula.

Alunos usando IA podem apresentar soluções que funcionam, mas talvez usem técnicas não ensinadas (por exemplo, uma chamada de API ou uma biblioteca não permitida). Nesses casos, verifique se o aluno **sabe justificar** essa escolha diferente (retomando o ponto das justificativas técnicas). Se não souber, isso impacta a avaliação. Resumindo, usar IA não exime o estudante de escrever código limpo e bem documentado, pelo contrário, espera-se que ele **revise e aprimore** o código sugerido. Assim, na avaliação, dê crédito àquele que claramente **fez curadoria do código** (organizou, nomeou, comentou) em contraste ao que talvez apenas colou a saída bruta da IA sem polimento.

9.8 Autoria e verificação de entendimento individual

Por fim, um componente crucial da avaliação diferenciada é confirmar **quem é o autor intelectual do projeto e se ele domina o conteúdo**. Como visto, métodos automáticos de detecção de plágio ou similaridade são menos eficazes com geração de IA, porque o código pode ser original nos caracteres, mas não fruto da aprendizagem do aluno. Portanto, os professores devem lançar mão de estratégias ativas para validar a autoria. Uma das mais eficientes é a **defesa oral ou demonstração prática**: reserve um tempo para cada estudante (ou duplas, se for projeto em grupo) apresentar seu trabalho, rodar o programa e responder a perguntas.

Durante essa conversa, explore trechos específicos do código: “o que faz esta função? Como você a implementou?”, “por que você escolheu essa abordagem e não aquela outra?”, ou até “se eu quisesse adicionar tal funcionalidade, por onde você começaria?”. A forma como o aluno responde, com segurança e detalhes, ou com hesitação e generalidades, fala volumes sobre o nível de envolvimento dele com o código. Muitos alunos, sabendo que haverá uma arguição, **pensam duas vezes antes de entregar algo que não entendem**, o que tem efeito preventivo.

Outra tática é propor um **pequeno desafio relacionado durante a apresentação**: por exemplo, pedir que altere uma cor na interface, ou que corrija um defeito simples que você propositalmente insere. Novamente, quem programou saberá rapidamente onde mexer, enquanto quem não tem familiaridade vai travar. Em turmas grandes, a defesa individual de todos pode ser inviável; mas é possível selecionar aleatoriamente um subconjunto para verificar (avisando os alunos disso) ou mesmo aplicar **provas práticas curtas** em laboratório com problemas similares. Além da oralidade, **analisar o histórico de desenvolvimento** é útil: se o trabalho foi desenvolvido em repositório Git, **verifique os commits**. Commits progressivos ao longo de dias/semanas, com mensagens coerentes, sugerem construção própria; já um repositório com um único commit adicionando milhares de linhas às vésperas do prazo indica possível importação externa. Você pode pedir explicitamente que os alunos entreguem o link do repositório ou um arquivo compactado incluindo um log de versões.

Outra alternativa é fracionar a entrega: primeiro entrega-se o planejamento (esboço, diagrama, lista de tarefas), depois uma versão parcial, depois a final. Tudo isso dificulta que alguém deixe para “pedir tudo à IA” de última hora. Em essência, a mensagem ao estudante deve ser: “eu valorizo o seu aprendizado, não apenas o produto final; por isso vou conversar com você sobre o que fez”. Essa postura assegura que, mesmo que parte do código venha a ser IA-assistida, **a autoria intelectual e o entendimento sejam do aluno**, pois ele precisará explicar e reproduzir o resultado. Com essas salvaguardas, o uso da IA torna-se um aliado (o aluno pode acelerar partes do desenvolvimento) e não um atalho antiético (já que ele será chamado a demonstrar domínio de qualquer forma).

Com os métodos acima combinados, é possível montar um esquema de avaliação robusto e justo mesmo na era das IAs de programação. Por exemplo, um projeto poderia ser avaliado assim: 50% pela funcionalidade/código entregue (onde o aluno pode ter usado IA, tudo bem), 20% por um relatório explicativo e reflexivo sobre o código e uso da ferramenta, 15% por uma apresentação com perguntas (ou vídeo explicativo), 15% por um pequeno teste individual conceitual relacionado. Cada componente ataca um ângulo: produto, processo, comunicação e teoria. Essa abordagem múltipla **dificulta fraudes** e, mais importante, **incentiva o aluno a realmente aprender** para conseguir cumprir todas as partes.

Em síntese, avaliar na presença de ferramentas de IA exige criatividade, mas também oferece a chance de **avaliar de forma mais completa**: não apenas o resultado final, mas a compreensão, a capacidade de explicar, de adaptar e de refletir. Essas habilidades são exatamente as que distinguem um profissional bem formado. E, de quebra, a avaliação diferenciada comunica aos estudantes que o professor valoriza o entendimento sobre a nota, desencorajando a dependência cega na IA (já que ela, sozinha, não garantirá sucesso em todas as etapas avaliativas).

10 ORIENTAÇÕES E BOAS PRÁTICAS PARA PROFESSORES

A incorporação de ferramentas de IA como Replit e Lovable no ensino exige também que o **professor** adapte suas práticas. A seguir, listamos algumas orientações e sugestões de boas práticas para educadores navegarem esse novo cenário, equilibrando inovação com controle pedagógico.

10.1 Mantenha-se atualizado e experimente as ferramentas

Antes de cobrar ou esperar que os alunos usem conscientemente a IA, é importante que o professor tenha familiaridade com elas. Dedique tempo para você mesmo explorar o Replit (especialmente o Ghostwriter/Agent) e o Lovable, entendendo seus pontos fortes e limitações. Tente gerar um projeto com cada um, veja que tipo de saída produzem, onde costumam errar, quanto demandam de intervenção humana. Isso permitirá que você **oriente com propriedade**, por exemplo, sabendo que o Ghostwriter tende a certo tipo de solução ou que Lovable configura a aplicação de determinada forma, você pode antecipar dúvidas dos alunos ou até preparar material de apoio (“quando a IA fizer X, prestem atenção em Y”).

Além disso, acompanhe as atualizações: esse campo evolui rápido. Blogs oficiais (como o da Replit) e comunidades (Discord, fóruns) geralmente anunciam novas funções ou mudanças. Por exemplo, se amanhã o Ghostwriter passar a comentar o código automaticamente, isso muda a dinâmica; se o Lovable incluir suporte a uma nova linguagem, pode interessar a certos cursos. Estar atualizado ajuda a **tomar decisões embasadas** sobre quando e como usar cada ferramenta. Também teste outras ferramentas similares (GitHub Copilot, Cursor etc.) para entender o panorama, mesmo que você escolha focar em Replit/Lovable, ter visão geral permite discutir de forma mais rica o papel da IA na programação com seus alunos.

10.2 Estabeleça políticas claras no plano de ensino

Transparência é fundamental. Logo no início da disciplina (e no plano/documento do curso), comunique explicitamente qual é a política de uso de IA nas avaliações e tarefas. Por exemplo: “nesta disciplina, o uso de ferramentas de geração de código por IA é permitido/não permitido nas seguintes situações...”, detalhando onde podem usar livremente (talvez nos projetos em casa), onde é restrito (provas fechadas) e quais obrigações vêm junto (como citar o uso, ou entregar explicações adicionais).

Se possível, alinhe isso com a coordenação do curso ou outros colegas, para que os alunos não recebam mensagens conflitantes em disciplinas diferentes. Como a questão é relativamente nova, pode não haver uma política institucional ainda, então seja pioneiro, mas documente tudo. Deixe claro que **uso indevido será tratado como qualquer outra forma de desonestidade acadêmica**, mas defina o que é indevido: por exemplo, “submeter código gerado automaticamente sem declarar a fonte ou sem ter condições de explicá-lo será considerado plágio/fraude”.

Essa clareza tende a inibir comportamentos inadequados, pois os alunos saberão que o professor está ciente dessas ferramentas e as leva a sério. Também enfatize no discurso que **a intenção não é punir quem usa IA para aprender, e sim garantir a justiça e efetividade do aprendizado**. Assim, os bons alunos não terão receio de aproveitá-las da forma correta.

10.3 Promova discussões éticas e reflexivas em sala

Aproveite o assunto para engajar a turma em debates. Por exemplo, proponha uma discussão: “se você usar o ChatGPT para fazer um trabalho, isso é considerado cola? Por quê?” ou “código gerado pela IA deve ter crédito no comentário? Afinal, quem é o autor?”. Essas conversas, moderadas pelo professor, ajudam os estudantes a **refletirem sobre autoria, originalidade e aprendizagem**. Você pode trazer exemplos do mundo real (casos noticiados de gente que usou IA inadequadamente, ou artigos argumentando sobre IA e plágio) para embasar. O importante é não fingir que a IA não existe nem proibi-la sem discussão. Deixe os próprios alunos ponderarem vantagens e desvantagens. Muitas vezes, eles chegam por conta própria à conclusão de que depender só de IA pode ser uma armadilha para eles mesmos. Incluir essa pauta também mostra que você, como professor, se preocupa não apenas com o conteúdo técnico, mas com a **formação ética e crítica** deles enquanto futuros profissionais.

Outra ideia: se houver oportunidade, envolva disciplinas ou colegas de áreas de Humanas/Ética para enriquecer a conversa. Em resumo, traga a IA como tema transversal, afinal, desenvolvedores conscientes não se formam só sabendo sintaxe, mas entendendo o contexto e consequências de suas ferramentas.

10.4 Use as ferramentas para apoio docente também

Lembre-se de que a IA pode ser aliada do professor no preparo de aulas e materiais. Por exemplo, o ChatGPT (ou o próprio Ghostwriter) pode ajudar a criar esboços de exercícios, gerar conjuntos de dados fictícios para problemas, corrigir rapidamente códigos dos alunos ou até propor sugestões de feedback.

Há estudos mostrando que o ChatGPT pode agilizar tarefas de preparo de avaliações, correção e fornecimento de feedback personalizado. Claro, deve-se revisar tudo (a IA do professor também erra!), mas isso pode economizar tempo. Por exemplo, você pode pedir ao ChatGPT: “sugira 5 variações de um problema de programação sobre árvore binária com níveis variados de dificuldade”, e então adaptar as melhores. Ou “dada esta resposta de aluno, gere um feedback comentando erros e acertos”, e usar como base. Ao automatizar o que for possível, você libera mais tempo para o que a IA não faz: **mentorar individualmente, pensar em intervenções pedagógicas, atualizar conteúdo**.

Além disso, ao usar as ferramentas, você se sensibiliza para como elas poderiam ser utilizadas pelos alunos. Por exemplo, se você mesmo usar o Ghostwriter para corrigir um bug de um exercício, vai perceber “puxa, ele deu a resposta de cara; na próxima vou formular o exercício de forma diferente ou pedir também a explicação do bug”. Em outras palavras, usar a IA a seu favor ajuda a calibrar o ensino. Só tenha cuidado de não se tornar você dependente (a ponto de aceitar respostas ou sugestões sem validar, o que seria análogo ao aluno preguiçoso). Use como um assistente, sempre com seu olhar crítico final, tal como quer que os estudantes façam.

10.5 Prepare-se para dar suporte diferenciado

Em uma turma, haverá alunos que se adaptam rápido e usam a IA com maestria e outros que ficam perdidos ou tentados a abusar. Esteja pronto para **identificar essas diferenças e intervir conforme necessário**. Os fortes você pode desafiar mais: por exemplo, se um aluno já domina o Ghostwriter, proponha que ele tente usar a API da Replit ou experimente ferramentas diferentes, mantendo-o engajado. Já com os que estão dependentes de mais, às vezes uma conversa particular ajuda: “notei que seu trabalho parece muito genérico, você usou alguma ferramenta? Vamos analisar juntos para você entender melhor.”, em vez de acusatório, seja orientador. Talvez esse aluno precise de reforço em fundamentos que pulou.

Em alguns casos, pode ser útil criar **pequenos tutoriais em vídeo** ou demonstrações de como usar a ferramenta responsavelmente, e disponibilizar no ambiente virtual da disciplina. Assim, quem tiver dificuldade pode rever. Considere também envolver a monitoria ou colegas: por exemplo, se há um grupo de alunos experientes, incentive a troca de dicas de uso de IA no fórum da turma. Quando os alunos ensinam entre si (como “olha, pedi tal coisa pro Ghostwriter e veio errado, então mudei o prompt de tal jeito e deu certo”), eles mesmos internalizam as lições. Em suma, o professor se torna um **facilitador** do aprendizado com IA, guiando cada aluno a tirar o melhor proveito sem se prejudicar.

10.6 Redesenhe atividades valorizando criatividade e originalidade

Uma boa prática para driblar a facilidade da IA resolver problemas padrão é **moldar atividades que peçam um toque pessoal ou criativo** dos alunos. Por exemplo, em vez de dar um enunciado muito genérico (“implemente algoritmo X”), contextualize num mini-cenário, peça para integrar com algo do mundo real, ou deixe margem para escolhas de design. A IA se sai melhor com problemas típicos e bem especificados; quando há espaços em aberto, o aluno precisa pensar.

Uma forma de desestimular soluções completamente geradas pela IA é **aumentar a complexidade ou o ineditismo dos projetos propostos**. Problemas muito comuns ou de baixo nível podem frequentemente ser resolvidos por modelos como o ChatGPT ou Copilot com poucos prompts. Portanto, convém propor desafios que exijam **multiestágios, integração de conhecimentos ou criatividade**. Por exemplo, em vez de pedir simplesmente “uma calculadora em Python”, pode-se demandar um aplicativo completo que realize cálculos, armazene histórico em arquivo e tenha uma interface gráfica básica, combinando diferentes tópicos.

Exemplo: ao invés de “faça um CRUD de biblioteca”, peça “desenvolva um minissistema para gerenciar os livros de uma biblioteca comunitária, considerando que futuramente poderá ter recurso de doação de livros, projete o código de forma extensível”. Isso requer do aluno pensar em extensibilidade (a IA não sabe do contexto futuro a menos que o prompt inclua, mas se incluir fica muito complexo).

Projetos mais abertos e com espaço para decisões de design forcem o aluno a ir além do que a IA normalmente sugere. Uma boa prática é o professor **testar previamente a tarefa com uma IA** para

avaliar se a solução sai muito trivial; caso sim, deve-se refinar os requisitos. Se o ChatGPT consegue gerar quase todo o código pedido de primeira, talvez seja necessário incluir requisitos adicionais, cenários de entrada menos usuais ou elementos de criatividade para elevar o nível de dificuldade. Essa estratégia se justifica porque já está comprovado que **muitos problemas introdutórios típicos podem ser resolvidos pela IA com alto grau de sucesso.**

Outro exemplo: combine programação com escrita, “codifique tal coisa e escreva um breve manual de uso para um usuário leigo”. A IA pode ajudar no código e até no manual, mas encaixar isso no escopo didático e revisar provavelmente fará o aluno aprender no processo. Em projetos, dê espaço para o aluno escolher o tema ou alguma funcionalidade extra, isso gera variedade nas entregas, dificultando soluções prontas iguais para todos. Quando cada projeto tem algo único (porque o aluno personalizou), a IA não tem como dar um pacote pronto; o aluno terá que integrar pedaços e isso exige compreensão. Claro que, dependendo do nível, às vezes exercícios padronizados são necessários, aí entra a parte de monitorar e avaliar diferenciado como já dito. Mas sempre que possível, **incentive originalidade.** Isso forma também desenvolvedores mais criativos e com portfólios mais interessantes.

Em um estudo, o Codex obteve ~78% de pontuação em provas de programação de introdução, posicionando-se entre os melhores alunos da classe. Logo, **diferenciar a atividade** é crucial: fornecer conjuntos de dados únicos para cada aluno, propor projetos tematicamente distintos ou inserir uma problemática do mundo real (que envolva ambiguidades e decisões) ajuda a garantir que o estudante precise pensar e contribuir além de apenas pedir uma resposta pronta ao ChatGPT. Em síntese, avalie se o código entregue tem **elementos de originalidade e complexidade condizentes com o nível do curso.** Soluções demasiado simples, linearmente semelhantes ao que qualquer IA geraria, devem acender um alerta e podem motivar perguntas adicionais ao aluno.

10.7 Fortaleça a cultura de aprendizado contínuo e adaptabilidade

Por fim, transmita aos estudantes uma mensagem importante: “no mundo da tecnologia, ferramentas vêm e vão, o mais importante é aprender a aprender e a se adaptar.”. Hoje falamos de Ghostwriter e Lovable, amanhã serão outras IAs ou paradigmas. Desenvolver a capacidade de **se adequar a novos fluxos de trabalho** (como programar com um par de IA) é em si uma competência valiosa.

Nesse sentido, por mais trabalho que dê ao professor agora repensar estratégias, isso está preparando os alunos para um futuro onde, provavelmente, a colaboração humano-IA será lugar-comum no desenvolvimento de software. Reforce com eles que dominar a ferramenta não é “trapaça”, é parte da formação de um profissional atual, contanto que dominem também o fundamento. Quando os estudantes entendem que o papel da instituição de ensino não é bloquear a IA, mas sim ensiná-los a usá-la de modo a **ampliar seu entendimento e não diminuí-lo**, eles tendem a abraçar esse aprendizado de forma madura.

Em resumo, para os professores, a recomendação é **não ignorar nem combater cegamente a IA, mas abraçá-la com visão crítica e pedagógica.** Adaptação curricular, franqueza nas regras, busca de apoio em comunidade (compartilhe experiências com outros docentes!) e inovação nas propostas de aula são passos necessários. Esse cenário, apesar de desafiador, é também uma oportunidade de renovar práticas de ensino de programação que vinham sendo usadas há décadas, alinhando-as com as demandas e ferramentas do século XXI.

11 O PAPEL DA IA NA FORMAÇÃO DE DESENVOLVEDORES CONSCIENTES

A presença de IA na programação levanta uma questão de fundo: **que tipo de profissional queremos formar em nossos cursos de Computação?** Em última instância, o objetivo não é apenas ensinar uma linguagem ou tecnologia específica, mas sim preparar desenvolvedores capazes de pensar, aprender continuamente e agir eticamente em sua profissão. As ferramentas de IA, quando usadas corretamente, podem contribuir muito para esse objetivo, mas quando usadas de forma irrefletida, podem atrapalhar. Por isso, vale uma reflexão final sobre o papel da IA na formação de desenvolvedores conscientes, isto é, profissionais tecnicamente competentes, eticamente responsáveis e críticos quanto ao uso da tecnologia.

Em primeiro lugar, é importante reconhecer que **a colaboração com IA será parte integrante do desenvolvimento de software no futuro** (na verdade, já é no presente). Ignorar ou proibir totalmente essas ferramentas na formação seria desconectar os alunos da realidade que os aguarda. Um desenvolvedor consciente precisará saber conviver com “colegas” não-humanos, seja um Ghostwriter, Copilot, ChatGPT ou outros sistemas, tirando proveito de suas vantagens e mitigando seus riscos. Isso significa que nosso papel como educadores evolui para **ensinar a trabalhar em conjunto com a IA**. Autores chegam a comparar colocar um modelo como o Codex nas mãos de estudantes a **dar uma ferramenta elétrica a um aprendiz de carpinteiro**: ela pode agilizar e ampliar o que ele faz, mas se mal usada, pode causar acidentes ou resultados ruins. A ferramenta em si não é boa ou má; tudo depende de como e quando é utilizada, e de quão preparado está o usuário. Assim, formar um desenvolvedor consciente hoje implica **ensinar não só código, mas também o discernimento no uso de ferramentas automáticas**.

Uma oportunidade que surge é **refatorar o currículo** e as práticas pedagógicas para focar mais nas habilidades que as máquinas não substituem facilmente. Por exemplo, se antes gastávamos muito tempo de aula fazendo os alunos codificarem do zero funções de ordenação ou CRUDs simples (o que agora uma IA faz em segundos), podemos redirecionar esse tempo para discutir otimização, análise de complexidade, design de sistemas, experiência do usuário, segurança da informação, aspectos que exigem compreensão e pensamento crítico mais aprofundado.

De certa forma, a IA força a educação a **subir o nível**, evitando um ensino centrado em memorização ou em tarefas mecânicas. Ao mesmo tempo, libera espaço para **mais aprendizagem ativa**: projetos, estudos de caso reais etc., onde o aluno pode usar a IA para acelerar partes menos essenciais e concentrar energia intelectual no cerne do problema. Claro, isso deve ser feito com cuidado para que o básico não fique negligenciado, mas é uma chance de tornar o aprendizado mais alinhado às demandas reais. A IA generativa consegue resolver muitos problemas “de livro-texto”? Ótimo, então vamos propor problemas mais complexos, contexto mais rico, onde o valor humano se sobressaia.

Além disso, a integração da IA no aprendizado pode estimular qualidades como **criatividade, iniciativa e colaboração**. Ao contrário do medo inicial de que a IA tornaria os alunos apáticos, pode ocorrer o oposto: se bem orientados, eles ganham liberdade para experimentar ideias novas (já que o custo de tentar e errar diminuiu, a IA ajuda a refazer rápido), para buscar conhecimento além do fornecido em aula (dialogando com a IA sobre assuntos relacionados) e para colaborar entre si discutindo as sugestões que receberam. Isso, obviamente, requer que o professor plante essa cultura e não apenas entregue a ferramenta sem orientação. Mas, quando o ambiente é propício, os estudantes passam a ver a IA como **mais um recurso de aprendizado**, tal como livros, Internet, tutores etc.

Por exemplo, um aluno curioso pode pedir ao ChatGPT explicações extra sobre um conceito visto em aula e trazer para o professor: “olha, a IA me explicou assim, isso procede?”. Esse tipo de atitude

mostra engajamento crítico, ele não apenas aceitou, ele validou com o professor. Estamos então formando alguém que sabe buscar informação e validar fontes, em vez de alguém que decora passivamente.

É fundamental também abordar a **responsabilidade ética** no uso de IA. Desenvolvedores conscientes precisarão lidar com dilemas como privacidade de dados (afinal, enviar código para uma IA de terceiro pode expor informações, isso deve ser considerado), vieses algorítmico (IA pode conter vieses nos exemplos que sugere, e o programador deve estar atento para não reproduzi-los), impactos sociais (usar IA aumenta produtividade, mas e as implicações no emprego de programadores? Devemos discutir isso com os futuros profissionais). Trazer esses debates durante a formação cria profissionais mais preparados para tomar decisões responsáveis.

Por exemplo, um aluno que entende que código gerado por IA pode vir de um repositório open-source com licença restritiva pensará duas vezes antes de simplesmente copiar e colar em um produto comercial, ele foi treinado para ser consciente das questões legais. Outro que discutiu em aula sobre dependência tecnológica excessiva vai saber equilibrar uso de AI com desenvolvimento das próprias skills. Essa consciência ampliada é o que queremos.

Por fim, o uso de IA no ensino pode contribuir para reposicionar o professor como um **mentor e facilitador**, e não apenas um transmissor de conteúdo. Com a IA cobrindo certos aspectos, o professor pode dedicar mais tempo a diálogos individuais, orientação de projetos, discussão de estratégias de solução, ou seja, atuar onde a experiência humana faz diferença. Isso vai ao encontro de teorias educacionais modernas (construtivismo, metodologias ativas) que já pregam essa mudança há tempos. A IA pode cuidar de “corrigir 100 códigos similares” ou “gerar exemplos triviais”, enquanto o professor foca em **feedback qualitativo, incentivo, mediação de debates, inspiração**. Essa mudança de postura enriquece a formação, pois os alunos têm mais acompanhamento personalizado no que realmente importa. Desenvolvedores conscientes muitas vezes creditam seu crescimento não apenas ao que aprenderam, mas a como aprenderam, e a figura de um professor-mentor faz diferença aí.

Em conclusão, o papel da IA na formação de desenvolvedores conscientes é **duplo**: por um lado, é uma ferramenta poderosa que, se usada eticamente, potencializa o aprendizado e aproxima o ensino da prática moderna; por outro, é um tema educativo em si, desafiando-nos a ensinar não só conteúdo técnico, mas também literacia digital, ética profissional e pensamento crítico sobre tecnologia. Como disse um grupo de pesquisadores, colocar IA nas mãos dos estudantes é como dar um “power tool”, e cabe a nós assegurar que eles aprendam a construir com ela, e não a se machucar ou trapacear.

Se conseguirmos isso, formaremos desenvolvedores aptos a **colaborar com a IA** de forma inteligente, sabendo que seu valor não foi substituído pela máquina, mas sim elevado porque eles dominam tanto a base humana quanto o recurso tecnológico. Esses profissionais terão vantagem num mundo em que a **competência** será medida não pela capacidade de digitar rápido um código (a IA faz isso), mas pela capacidade de resolver problemas, inovar, garantir qualidade e agir com responsabilidade.

12 QUALIDADE E SEGURANÇA DO CÓDIGO GERADO POR IA

É importante conscientizar os alunos de que **nem tudo que reluz é ouro**: o código produzido pelo ChatGPT pode até funcionar em casos simples, porém **geralmente carece de qualidade, boas práticas e segurança**, o que impacta seriamente a manutenção futura do software. Estudos recentes sustentam essa afirmação. Autores analisaram em larga escala trechos de código reais gerados pelo ChatGPT e confirmaram que, sem orientação explícita, **os modelos frequentemente produzem**

soluções abaixo do padrão mínimo de segurança, vulneráveis a ataques. Em particular, há uma **tendência alarmante de escolhas de implementação inseguras**: por exemplo, optar por concatenar strings em consultas de banco de dados (sujeitando-se a SQL Injection) quando não instruído a priorizar métodos seguros.

Esse comportamento foi observado em **cerca de 45% das tarefas de programação testadas**, ou seja, quase metade das vezes em que modelos de IA geram código, introduzem alguma vulnerabilidade de segurança grave. Outros experimentos revelaram falhas especialmente frequentes em sanitização de entrada: em mais de **86% dos casos** analisados, o código de IA não preveniu ataques de cross-site scripting ou de injeção de logs, por exemplo. Tais resultados mostram que o ChatGPT **prioriza entregar um código funcional e sintaticamente correto, mas não necessariamente seguro**, já que seus modelos aprendem de inúmeros exemplos da Internet, muitos dos quais funcionais, porém inseguros, e sem rótulos que os distingam.

Além de falhas de segurança, há **problemas de qualidade e manutenibilidade**. Segundo pesquisas, o código gerado pelo ChatGPT costuma apresentar **“code smells”** e descuidos que um bom programador evitaria. Entre os defeitos comuns estão: uso de **variáveis não definidas ou nunca utilizadas**, tratamento inadequado de exceções e recursos (exemplo: abrir arquivos ou conexões sem fechar), e **documentação deficiente** (comentários genéricos ou ausentes). Esses vícios dificultam a depuração e a evolução do código, tornando-o frágil em produção. Não por acaso, o estudo descobriu que **poucas contribuições de código gerado por IA são aceitas diretamente nos repositórios de desenvolvedores**, e mesmo quando o são, **passam por modificações significativas pelos programadores antes do merge**. Ou seja, profissionais frequentemente precisam **“consertar” ou refatorar extensamente** a saída da IA para adequá-lo aos padrões exigidos. Isso indica que **a dependência irrestrita dessas ferramentas pode levar a um déficit de manutenção no futuro**: sistemas construídos com muitos trechos gerados automaticamente podem acumular dívidas técnicas, vulnerabilidades e inconsistências que demandarão retrabalho pesado.

Como é identificado, à medida que trechos de código de baixa qualidade gerados por LLMs entram em produção, eles **prejudicam a segurança e a confiabilidade do software a longo prazo**. Portanto, do ponto de vista pedagógico, é fundamental alertar os alunos de que **usar o ChatGPT como atalho evita o esforço imediato, mas cobra seu preço depois**, seja em pontos perdidos por um bug oculto, seja em horas gastas tentando dar manutenção em um código confuso que eles próprios não compreendem bem.

13 USO DE IA GENERATIVA NA INDÚSTRIA DE SOFTWARE: COMO AS EMPRESAS ESTÃO LIDANDO
Enquanto escolas debatem restrições, **no mercado de trabalho a IA generativa vem sendo adotada de forma acelerada**, transformando práticas de desenvolvimento de sistemas. **Grandes empresas de tecnologia já utilizam intensivamente ferramentas como GitHub Copilot, ChatGPT ou modelos proprietários para auxiliar na codificação**. Sundar Pichai, CEO da Google, revelou que **mais de 25% do novo código na Google já é gerado por inteligência artificial**, embora sempre **revisto e aprovado por engenheiros humanos** posteriormente. Satya Nadella, da Microsoft, relatou percentual semelhante, dizendo que **20–30% do código nos repositórios da empresa já é escrito por IA**. Ou seja, nas maiores organizações a IA virou uma parceira de programação, elevando a produtividade em tarefas repetitivas e permitindo que desenvolvedores foquem em decisões de alto nível.

Há até previsões audaciosas no setor: a startup **Anthropic projeta que até setembro de 2025 cerca de 90% do código em empresas de tecnologia poderá ser gerado por IA**. Ainda que esse número

seja especulativo, ilustra a expectativa de uma **grande mudança de paradigma no desenvolvimento de software** nos próximos anos.

Diante desse cenário, as empresas estão buscando **equilibrar os benefícios da IA com a garantia de qualidade e segurança**. Muitas adotaram políticas internas para regular o uso de ferramentas generativas. Por exemplo, **Apple, JPMorgan, Samsung e outras corporações proibiram seus funcionários de usar o ChatGPT (e serviços similares) em projetos internos confidenciais**, devido a preocupações com vazamento de propriedade intelectual e dados sensíveis. Essas companhias temem que ao inserir código proprietário em um prompt de IA na nuvem, essas informações possam ser expostas ou mesmo incorporadas ao modelo. Em vez disso, algumas desenvolvem **soluções de IA internas e controladas** (como o código-assistente **Amazon CodeWhisperer** ou o **IBM WatsonX**) para colher os frutos da automação sem comprometer a segurança dos dados. Por outro lado, muitas organizações sinalizam que **os ganhos de produtividade superam os riscos**, se medidas de precaução forem tomadas.

Uma pesquisa da **Gartner em 2023** indicou que a maioria dos executivos acredita no balanço positivo dessas ferramentas, e cerca de **1 em cada 5 empresas já tinha pilotos avançados ou sistemas em produção usando IA generativa** em suas rotinas. Assim, ao invés de banir totalmente, várias empresas optam por **diretrizes claras e treinamento**: ensinando os desenvolvedores a usar a IA de forma responsável (por exemplo, **não confiar cegamente** e sempre revisar a saída da máquina) e **integrando verificações automatizadas no pipeline**.

De fato, **boas práticas de engenharia de software estão sendo adaptadas para a era da IA**. Companhias de segurança recomendam implementar **ferramentas de análise estática e dinâmica nos ciclos de desenvolvimento para inspecionar todo código gerado por IA**, impedindo que vulnerabilidades passem despercebidas. No relatório “GenAI Code Security 2025” da Veracode, enfatiza-se que organizações precisam **prevenir falhas antes que cheguem à produção**, incorporando desde cedo verificações de qualidade do código e correções automatizadas no fluxo de trabalho de programadores. Em outras palavras, as empresas estão ajustando suas pipelines CI/CD para que a IA seja uma aliada sob supervisão: **o código sugerido pela máquina passa por rigorosas revisões humanas e por scanners de vulnerabilidades**, garantindo que somente soluções seguras e sustentáveis sejam implantadas.

Outra frente de ação é **o desenvolvimento de competências e cultura**. Em vez de enxergar a IA como ameaça, muitas empresas esperam que seus funcionários **dominem essas novas ferramentas** para aumentar eficiência. Segundo reportagem do Wall Street Journal, **25% das empresas de tecnologia já buscam ativamente contratar profissionais com experiência em ferramentas de IA**, reconhecendo seu valor estratégico. O próprio Sam Altman, CEO da OpenAI, destacou que **programar continuará importante, mas saber usar bem a IA será tão essencial quanto** nos próximos anos. Portanto, a tendência nas organizações é **conscientizar e capacitar desenvolvedores** para trabalharem lado a lado com a IA, sem abdicar do senso crítico. Eles devem aprender a avaliar as sugestões do ChatGPT, ajustar prompts para obter respostas melhores e verificar exaustivamente a correção e segurança antes de incorporar qualquer trecho. As empresas pioneiras nesse equilíbrio relatam que a IA pode, sim, **aumentar a velocidade de codificação e até reduzir pequenos erros** ao sugerir sintaxes corretas. Porém, elas deixam claro que **a responsabilidade final permanece com o desenvolvedor humano**, este precisa garantir que o código faz o que deve fazer de forma **correta, eficiente, segura e mantível**.

14 CHECKLIST DOS ESTUDANTES PARA USO CORRETO DE IA NA PROGRAMAÇÃO

Na sequência, apresentamos um checklist para orientar estudantes a utilizar IA como apoio ao raciocínio na programação, mantendo autoria, compreensão do código, segurança e responsabilidade acadêmica.

a) Antes de usar a IA

- ☐ Eu compreendi o problema antes de pedir ajuda à IA.
- ☐ Tentei estruturar a solução (fluxograma, pseudocódigo ou rascunho lógico).
- ☐ Sei exatamente qual parte do problema quero que a IA me ajude a resolver.
- ☐ Verifiquei se o uso da IA é permitido nesta atividade (de acordo com as regras do professor).

b) Ao criar o prompt

- ☐ Meu pedido está claro e específico (linguagem, objetivo, restrições).
- ☐ Não pedi simplesmente “faça tudo para mim”, mas sim ajuda em partes específicas.
- ☐ Quando possível, solicitei explicações junto com o código.
- ☐ Evitei compartilhar dados sensíveis, informações privadas ou segredos comerciais (propriedade intelectual) no prompt.

c) Após receber o código

- ☐ Li todo o código com atenção antes de usar.
- ☐ Entendo o que cada função, variável e estrutura faz.
- ☐ Inspecionei o código em busca de falhas de segurança comuns (como SQL Injection ou XSS) e erros de sanitização.
- ☐ Verifiquei a presença de “code smells”, como variáveis não utilizadas, conexões não fechadas ou tratamento inadequado de exceções.
- ☐ Testei o código com diferentes entradas.
- ☐ Verifiquei se há erros, inconsistências ou comportamentos inesperados.
- ☐ Analisei se o código realmente resolve o problema proposto.
- ☐ Pesquisei trechos ou bibliotecas que eu não conhecia.

d) Ao integrar no projeto

- ☐ Adapte o código ao meu contexto, em vez de apenas copiar e colar.
- ☐ Ajustei nomes de variáveis, comentários e organização para manter coerência.
- ☐ Garanti que o código esteja alinhado com os requisitos do projeto.
- ☐ Refatorei o código para evitar “dívida técnica” e problemas de manutenção futura.
- ☐ Realizei testes adicionais após modificar o código.

e) Sobre autoria e ética

- ☐ Declarei o uso da IA quando necessário.
- ☐ Consigo explicar todo o código entregue, mesmo que tenha sido gerado pela IA.
- ☐ Não utilizei a IA para substituir completamente meu processo de aprendizagem.
- ☐ Estou usando a ferramenta como apoio, não como atalho para evitar esforço.

f) Validação do meu aprendizado

- ☐ Se o professor pedir, consigo modificar o código em tempo real.
- ☐ Consigo justificar as escolhas de arquitetura ou lógica adotadas.
- ☐ Sei reescrever partes do código sem consultar a IA.
- ☐ Consigo implementar algo semelhante sem depender totalmente da ferramenta.

g) Uso crítico e profissional

- ☐ Questionei se a solução gerada é eficiente e segura.
- ☐ Avaliei possíveis problemas de desempenho ou vulnerabilidades.
- ☐ Avaliei se o código precisaria de modificações significativas para ser aceito em um repositório profissional (code review).
- ☐ Verifiquei se não há dependência excessiva de soluções prontas.
- ☐ Usei a IA para aprender algo novo, não apenas para entregar a tarefa.

h) Pergunta final de autoverificação

- ☐ Se alguém me perguntar como meu código funciona, eu consigo explicar com segurança e clareza? Se a resposta for “não”, ainda não estou usando a IA de forma adequada.

15 CHECKLIST PARA PROFESSORES AVALIAR TRABALHOS DESENVOLVIDOS COM APOIO DE IA

A seguir, é apresentado um checklist objetivo para docentes analisarem atividades produzidas com apoio de IA, considerando autoria, coerência conceitual, segurança e evidências de aprendizagem do estudante. O foco da avaliação não deve ser detectar se houve uso de IA, mas verificar se houve aprendizagem, domínio conceitual e desenvolvimento da autonomia.

a) Antes da atividade (planejamento docente)

- ☐ Defini claramente se o uso de IA é permitido, limitado ou obrigatório.
- ☐ Estabeleci critérios explícitos de transparência (exemplo: declaração de uso de IA).
- ☐ Ajustei os critérios de avaliação para incluir compreensão e não apenas funcionamento do código.
- ☐ Estruturei a atividade para exigir justificativas técnicas e reflexão.
- ☐ Preparei perguntas ou desafios adicionais para validação de autoria.
- ☐ Testei previamente se a IA gera soluções inseguras para o problema proposto (exemplo: SQL Injection) para alertar ou avaliar a correção pelos alunos.

b) Transparência e ética

- ☐ Solicitei que o aluno declare se utilizou IA e em quais partes.
- ☐ Exigi descrição dos prompts utilizados (quando pertinente).
- ☐ Orientei sobre limites éticos (plágio, cópia automática, dependência excessiva).
- ☐ Orientei explicitamente sobre a proibição de inserir dados sensíveis ou confidenciais em ferramentas de IA públicas.
- ☐ Deixei claro que o estudante deve ser capaz de explicar integralmente o código entregue.
- ☐ Promovi discussão prévia sobre uso responsável de IA.

c) Verificação de compreensão técnica

- ☐ Solicitei explicação oral ou escrita do funcionamento do código.
- ☐ Pedi justificativa das decisões de arquitetura, estrutura de dados ou padrões utilizados.
- ☐ Questionei escolhas de bibliotecas ou frameworks adotados.
- ☐ Verifiquei se o código possui falhas de segurança comuns (exemplo: falta de sanitização, injeção de logs) frequentemente geradas por IA.
- ☐ Verifiquei se o aluno compreende complexidade, desempenho e limitações da solução.
- ☐ Observei se o estudante consegue identificar possíveis melhorias.

d) Validação prática (autoria e domínio)

- ☐ Solicitei modificação pontual no código em tempo real.
- ☐ Propus pequena alteração de requisito após entrega inicial.
- ☐ Pedi implementação adicional sem uso de IA.
- ☐ Avaliei capacidade de depuração do próprio código.

- ☐ Verifiquei se o aluno consegue reproduzir parte da lógica manualmente.

e) Critérios de avaliação diferenciada

- ☐ Avaliei processo e não apenas produto final.
- ☐ Considerei clareza, organização e documentação do código.
- ☐ Analisei a presença de “code smells” (variáveis inúteis, recursos não fechados) típicos de geração automática.
- ☐ Avaliei a manutenibilidade do código para evitar acúmulo de dívida técnica.
- ☐ Analisei coerência entre requisitos e implementação.
- ☐ Avaliei capacidade de argumentação técnica.
- ☐ Incluí componente reflexivo na nota final.

f) Avaliação escrita

- ☐ Solicitei relatório explicando o raciocínio por trás da solução.
- ☐ Pedi análise crítica sobre uso da IA (benefícios e limitações percebidas).
- ☐ Exigi comparação entre solução gerada e possíveis alternativas.
- ☐ Avaliei qualidade técnica da documentação produzida.

g) Avaliação oral

- ☐ Realizei apresentação ou defesa técnica do projeto.
- ☐ Fiz perguntas imprevisíveis sobre o funcionamento interno.
- ☐ Solicitei explicação de trechos específicos do código.
- ☐ Observei segurança, clareza e domínio conceitual do aluno.

h) Avaliação formativa (durante o processo)

- ☐ Acompanhei versões intermediárias do projeto.
- ☐ Solicitei entregas parciais antes do produto final.
- ☐ Estimulei registro de decisões técnicas ao longo do desenvolvimento.
- ☐ Dei feedback sobre qualidade do raciocínio e não apenas sobre erros.

i) Formação ética e profissional

- ☐ Promovi debate sobre autoria, propriedade intelectual e responsabilidade.
- ☐ Expliquei riscos de uso acrítico de código gerado por IA.
- ☐ Discuti como a indústria revisa rigorosamente o código de IA antes de integrá-lo a repositórios oficiais.
- ☐ Incentivei postura de coautoria, não de dependência.
- ☐ Valorizei pensamento crítico e autonomia técnica.

j) Boas práticas institucionais

- ☐ Alinhei critérios de avaliação com o colegiado do curso.
- ☐ Registrei diretrizes claras no plano de ensino.
- ☐ Evitei decisões punitivas sem discussão pedagógica prévia.
- ☐ Promovi cultura de transparência em vez de vigilância.
- ☐ Atualizei estratégias avaliativas conforme maturidade da turma.

k) Indicadores de alerta (sinais de uso acrítico)

- ☐ Incapacidade de explicar o código entregue.
- ☐ Dificuldade em realizar pequenas modificações.
- ☐ Uso excessivo de estruturas não estudadas em aula sem justificativa.
- ☐ Presença de vulnerabilidades de segurança graves (exemplo: concatenação de strings em SQL).

- ☐ Código com tratamento inadequado de exceções ou documentação genérica.
- ☐ Incoerência entre discurso técnico e implementação.

I) Pergunta-chave para o professor

- ☐ Este aluno demonstrou compreensão real do que foi desenvolvido, independentemente do uso de IA? Se a resposta for “sim”, o objetivo pedagógico foi atingido.

16 CONCLUSÃO

As plataformas de geração de código por IA como Replit e Lovable vieram para ficar e já estão redesenhando o cenário do ensino de programação. Em vez de evitá-las, a educação deve **abraçá-las com consciência**, integrando-as como aliadas no processo formativo. Este guia buscou mostrar que é possível, e desejável, usar tais ferramentas de modo pedagógico e ético, extraíndo seus benefícios (suporte à lógica, exemplos instantâneos, prototipagem rápida) e contornando seus perigos (plágio, aprendizado superficial, dependência). Para isso, alunos e professores precisam atuar em parceria: os alunos tornando-se coautores responsáveis, sempre engajados em compreender e criticar o código gerado; e os professores, mediando essa interação, ajustando métodos de ensino e avaliação para valorizar o entendimento e a integridade.

Em sala de aula, isso se traduz em novas dinâmicas, como análises de código assistido, desafios de melhoria, uso orientado de assistentes de IA e avaliações diferenciadas que priorizam explicação e adaptação. Fora da sala, implica criar uma cultura onde o aprendizado contínuo e ético é enfatizado, onde usar IA não é “cola”, mas, também, não é muleta; é uma habilidade a mais no repertório do futuro desenvolvedor.

O foco central permanece **na formação técnica, ética e crítica** dos estudantes. As ferramentas podem mudar, as linguagens evoluir, mas se ensinarmos nossos alunos a pensar, a aprender a aprender e a agir com honestidade, cumprimos nosso papel. A inteligência artificial, assim, deixa de ser vista como inimiga do ensino e se torna um catalisador para uma educação mais rica e atual.

Em última análise, formar desenvolvedores conscientes em tempos de IA é formar profissionais capazes de unir o melhor da inteligência humana: criatividade, senso crítico, empatia e responsabilidade, com o melhor da inteligência artificial: velocidade, precisão e vasto conhecimento disponível. Este equilíbrio, sem dúvida, será o alicerce das inovações e soluções do futuro. Cabe a nós, educadores e estudantes, construir juntos esse caminho, usando as novas ferramentas com sabedoria e ética.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **ABNT NBR ISO/IEC 42001**: tecnologia da informação: inteligência artificial: sistema de gestão. Rio de Janeiro: ABNT, 2024.

BECKER, Brett A. et al. Programming is hard — or at least it used to be: educational opportunities and challenges of AI code generation. In: **ACM Technical Symposium on Computer Science Education (SIGCSE)**, 54., 2023. Proceedings... New York: Association for Computing Machinery, 2023. p. 500–506. DOI: <https://doi.org/10.1145/3545945.3569759>.

CANDIDO, Lucas S.; BARBOSA, Christian A. de Melo; MARTINS, Lylia G.; COSTA, Esdras J. H. Análise de ferramentas de detecção de IA para textos científicos em português gerados por ChatGPT, Gemini e DeepSeek. In: Workshop Sobre as Implicações da Computação na Sociedade (WICS), 6., 2025, Maceió/AL. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2025. p. 78-91. DOI: <https://doi.org/10.5753/wics.2025.8692>.

CODESTRINGERS. **Replit Ghostwriter vs. Copilot: which is better?** Disponível em: <https://www.codestringers.com/insights/replit-ghostwriter-vs-copilot/>. Acesso em: 10 jan. 2026.

DEMPERE, J.; MODUGU, K.; HESHAM, A.; RAMASAMY, L. K. The impact of ChatGPT on higher education. **Frontiers in Education**, v. 8, 1206936, 2023. DOI: <https://doi.org/10.3389/feduc.2023.1206936>.

DENNY, Paul et al. Computing education in the era of generative AI. **Communications of the ACM**, Nova York, 2023. Disponível em: <https://doi.org/10.48550/arXiv.2306.02608>. Acesso em: 15 nov. 2025.

DUPONT, Gabriel Fontanive. **Uso do ChatGPT e as políticas de uso para sua empresa**. Publicado em: 19 jul. 2023. Disponível em: <https://www.dsfadogados.com.br/blog/uso-do-chatgpt-e-as-politicas-de-uso-para-sua-empresa>. Acesso em: 12 dez. 2025.

FAGUNDES, Tatiane. **O que é o ChatGPT e quais são os riscos para as empresas?** Publicado em: 27 mar. 2023. Disponível em: <https://netrin.com.br/blog/chatgpt-o-que-e-e-riscos/>. Acesso em: 12 dez. 2025.

FREITAS, Edvaldo. **Como código gerado por IA está impactando na dívida técnica**. Publicado em: 3 mar. 2025. Disponível em: <https://kodus.io/codigo-gerado-por-ia-esta-impactando-na-divida-tecnica/>. Acesso em: 19 dez. 2025.

FREITAS, Edvaldo. **Os maiores perigos do código gerado por IA (e como evitá-los)**. Publicado em: 11 fev. 2025. Disponível em: <https://kodus.io/perigos-codigo-ia/>. Acesso em: 19 dez. 2025.

FRENCH, Laura. **LLMs make insecure coding choices for 45% of tasks, study finds**. Publicado em: 30 jul. 2025. Disponível em: <https://www.scworld.com/news/llms-make-insecure-coding-choices-for-45-of-tasks-study-finds>. Acesso em: 16 dez. 2025.

GITLAB. **Explicação sobre a geração de código com IA: um guia para desenvolvedores**. 2025. Disponível em: <https://about.gitlab.com/pt-br/topics/devops/ai-code-generation-guide/>. Acesso em: 19 dez. 2025.

GOMES, Bruna. **Conheça os perigos do uso de IA no processo de desenvolvimento de aplicações**. Publicado em: 5 fev. 2025. Disponível em: <https://www.contacta.com.br/blog/conheca-os-perigos-do-uso-de-ia-no-processo-de-desenvolvimento-de-aplicacoes>. Acesso em: 19 dez. 2025.

GUROVA, Dora. **Replit vs Bolt vs Lovable: which AI coding tool should you use?** Publicado em: 23 abr. 2025. Disponível em: <https://uibakery.io/blog/replit-vs-bolt-vs-lovable>. Acesso em: 10 jan. 2026.

ISOTANI, Seiji et al. **ChatGPT pode ser aliado no processo de ensino-aprendizagem, avalia especialista. [Depoimento a Elton Alisson]**. São Paulo: Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo. Disponível em: <https://agencia.fapesp.br/chatgpt-pode-ser-aliado-no-processo-de-ensino-aprendizagem-avalia-especialista/40862/>. Acesso em: 02 jul. 2025.

KHANMOHAMMADI, Kobra et al. **WildCode**: an empirical analysis of code generated by ChatGPT. 2025. Disponível em: <https://arxiv.org/abs/2512.04259>. Acesso em: 15 dez. 2025.

LIRA, Werney Ayala Luz; SANTOS NETO, Pedro de A. dos; OSORIO, Luiz Fernando Mendes. Uma análise do uso de ferramentas de geração de código por alunos de computação. In: Simpósio Brasileiro de Educação em Computação (EDUCOMP), 4., 2024, Evento Online. **Anais [...]**. Porto Alegre: Sociedade Brasileira de Computação, 2024. p. 63-71. DOI: <https://doi.org/10.5753/educomp.2024.237427>.

MALWAREBYTES. **IA na segurança cibernética**: riscos da IA. 2025. Disponível em: <https://www.malwarebytes.com/pt-br/cybersecurity/basics/risks-of-ai-in-cyber-security>. Acesso em: 10 dez. 2025.

MARQUES, Fabrício. **O plágio encoberto em textos do ChatGPT**. Revista Pesquisa FAPESP, Publicado em: 9 mar. 2023. Disponível em: <https://revistapesquisa.fapesp.br/o-plagio-encoberto-em-textos-do-chatgpt/>. Acesso em: 19 dez. 2025.

MOVEO.AI. **Empresas que proíbem o ChatGPT (2025)**: lista e segurança. Publicado em: 15 dez. 2025. Disponível em: <https://moveo.ai/pt/blog/empresas-que-baniram-o-openai>. Acesso em: 10 dez. 2025.

OLIVEIRA, Vinícius. **IA na educação**: oportunidades e riscos além das telas. Porvir, 17 jul. 2025. Disponível em: <https://porvir.org/ia-educacao-oportunidades-riscos-alem-telas/>. Acesso em: 20 jul. 2025.

PERRY, Neil; SRIVASTAVA, Megha; KUMAR, Deepak; BONEH, Dan. Do users write more insecure code with AI assistants? In: **ACM SIGSAC Conference on Computer and Communications Security (CCS)**, 2023. Proceedings... New York: ACM, 2023. p. 2785–2799. DOI: <http://dx.doi.org/10.1145/3576915.3623157>.

PETERS, Jay. **More than a quarter of new code at Google is generated by AI**. Publicado em: 29 out. 2024. Disponível em: <https://www.theverge.com/2024/10/29/24282757/google-new-code-generated-ai-q3-2024>. Acesso em: 19 dez. 2025.

PINHEIRO, Petrilson; ARRUDA, Débora; PINHEIRO, Edson. O uso do ChatGPT como estratégia de avaliação na universidade: mobilizando os processos de conhecimento dos multiletramentos. **Raído**, v. 19, n. 48, p. 239–256, 2025. DOI: 10.30612/raido.v19i48.20310. Disponível em: <https://ojs.ufgd.edu.br/Raído/article/view/20310>. Acesso em: 17 fev. 2026.

PURYEAR, Ben; SPRINT, Gina. GitHub Copilot in the classroom: learning to code with AI assistance. **Journal of Computing Sciences in Colleges**, v. 38, n. 1, p. 37–47, out. 2022.

RAFTERY, Damien. Will ChatGPT pass the online quizzes? Adapting an assessment strategy in the age of generative AI. **Irish Journal of Technology Enhanced Learning**, v. 7, n. 1, 2023. DOI: 10.22554/ijtel.v7i1.114. Disponível em: <https://journal.ilta.ie/index.php/telji/article/view/114>. Acesso em: 10 nov. 2025.

SIDDIQ, Mohammed L. et al. Quality assessment of ChatGPT generated code and their use by developers. In: 21st Intl. Conference on Mining Software Repositories (MSR '24), Lisboa, 15–16 abr.

2024. New York: ACM, 2024. p. 1–5. Disponível em:
https://s2e-lab.github.io/preprints/msr_mining_challenge24-preprint.pdf. Acesso em: 12 dez. 2025.

SIQUEIRA, Ivan. **Regulação e limites éticos ao uso do ChatGPT na educação**. Disponível em:
https://www.youtube.com/watch?v=DEcR_afcBSM. Acesso em: 17 fev. 2026.

THE UNIVERSITY OF CHICAGO. **Strategies for leveraging GAI in course assignments**. Chicago, 2024. Disponível em:
<https://genai.uchicago.edu/en/resources/faculty-and-instructors/strategies-for-leveraging-gai-in-course-assignments>. Acesso em: 01 jul. 2025.

TORRES, Roberto. **Apple restricts ChatGPT, GitHub Copilot use over data worries**: report. Publicado em: 19 maio 2023. Disponível em:
<https://www.ciodive.com/news/apple-chatgpt-openai-copilot-generative-AI/650816/>. Acesso em: 16 dez. 2025.

UNESCO. **ChatGPT e inteligência artificial na educação superior**: guia de início rápido. [E. Sabzalieva; A. Valentini (org.)]. Paris: UNESCO IESALC, 2023. 14 p. Disponível em:
https://unesdoc.unesco.org/ark:/48223/pf0000385146_por. Acesso em: 10 nov. 2025.

VUKOJIČIĆ, Milić; KRSTIĆ, Jovana. **ChatGPT in programming education**: ChatGPT as a programming assistant. 2023.

XAVIER, Celso Niskier. **Uso da IA na educação terá regras pela primeira vez no Brasil**. Publicado em: 25 jul. 2025. Disponível em:
<https://valor.globo.com/empresas/noticia/2025/07/24/uso-da-ia-na-educacao-tera-regras-pela-primeira-vez-no-brasil.ghtml>. Acesso em: 29 jul. 2025.

YILMAZ, Ramazan; KARAOGLAN YILMAZ, Fatma Gizem. Augmented intelligence in programming learning: examining student views on the use of ChatGPT for programming learning. **Computers in Human Behavior: Artificial Humans**, v. 1, n. 2, p. 100005, 2023. DOI:
<https://doi.org/10.1016/j.chbah.2023.100005>. Disponível em:
<https://www.sciencedirect.com/science/article/pii/S2949882123000051>. Acesso em: 10 fev. 2026.

YILMAZ, Ramazan; KARAOGLAN YILMAZ, Fatma Gizem. The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. **Computers and Education: Artificial Intelligence**, v. 4, 100147, 2023. DOI:
<https://doi.org/10.1016/j.caeai.2023.100147>. Disponível em:
<https://www.sciencedirect.com/science/article/pii/S2666920X23000267>. Acesso em: 15 fev. 2026.

ZYBOOKS. **Ten strategies for preventing cheating in coding class**. 2023. Disponível em:
<https://www.zybooks.com/ten-strategies-to-preventing-cheating-in-coding-class-zybooks-guide/>. Acesso em: 12 dez. 2025.